

**ALGERIAN DEMOCRATIC AND
POPULAR REPUBLIC**



Ministry of Vocational Education and
Training



National Specialized Institute of Vocational
Training

Rahmania - Sidi Abdellah

Final Training Thesis

For the Obtaining of the Higher Technician Diploma
Web and Mobile Developer

Theme:

Design and Implementation of a Resource Assignment and Allocation
Platform for Djazzy — Case Study: IT and HR Departments

Host Organization:

Djazzy — Orascom Telecom Algeria

Supervisor:

Mme Krim

Promoter:

M. Ali Bouras

Presented by:

**Benkheira Mohamed Kamal
Brahimi Mohamed Amine**

Promotion: 2025/2026

Remerciements

We would like to express our sincere gratitude to the people who contributed to the completion of this final training thesis.

We thank our promoter **M. Ali Bouras** for his guidance, valuable advice, and continuous support throughout the development of this project. His expertise in software development and his availability made it possible to overcome the many technical challenges encountered during this work.

We also thank our academic supervisor **Mme Krim** from the institute for her follow-up, availability, and constructive feedback. Her methodological guidance was instrumental in structuring this report and ensuring the quality of our work.

We express our gratitude to our classmates for their solidarity and support throughout our training at INSFP Rahmania. The exchange of ideas and mutual assistance have been a valuable source of motivation.

Finally, we thank the staff of **Djezzy — Orascom Telecom Algeria** for welcoming us into their teams and providing us with the necessary resources to carry out this project under optimal conditions.

Dedicace

*To my family,
for their unwavering support and encouragement
throughout my studies.*

— *Benkheira Mohamed Kamal*

Dedicace

*To my parents,
for their sacrifices and unconditional love.
This achievement is yours as much as it is mine.*

— Brahim Mohamoud Amine

Resume

Ce travail porte sur la conception et la réalisation d'une plateforme web d'affectation et d'allocation des ressources pour Djezzy. L'objectif principal est de centraliser la gestion des profils employés, des compétences, des disponibilités et des charges de travail, tout en fournissant un workflow d'affectation complet avec traçabilité (audit trail). La solution utilise Laravel 13, Filament 5 et React/Inertia avec Tailwind CSS, et intègre une authentification renforcée (double facteur, clés d'accès).

Mots-clés : Allocation des ressources, Laravel, Filament, React, Spatie Permission, Audit trail

Abstract

This project involves the design and implementation of a web-based resource assignment and allocation platform for Djezzy. The main objective is to centralize employee profile management, skills tracking, availability monitoring, and workload distribution, while providing a complete assignment workflow with full audit traceability. The solution is built with Laravel 13, Filament 5, and React/Inertia with Tailwind CSS, and integrates strong authentication mechanisms (two-factor authentication, passkeys) via Laravel Fortify.

Keywords: Resource Allocation, Laravel, Filament, React, Spatie Permission, Audit Trail

المُلَخَّص

يتناول هذا العمل تصميم وتطوير منصة ويب لتخصيص وتوزيع الموارد لشركة Djezzy. الهدف الرئيسي هو توحيد إدارة ملفات الموظفين وتتبع المهارات ومراقبة التوفر وتوزيع عبء العمل، مع توفير سير عمل كامل لتخصيص الموارد مع التتبع الكامل للتدقيق. تم بناء الحل باستخدام 13 Laravel و 5 Filament و React/Inertia مع CSS، Tailwind ويتضمن آليات مصادقة قوية (المصادقة الثنائية ومفاتيح الوصول) عبر Fortify، Laravel.

الكلمات المفتاحية: تخصيص الموارد، Laravel، Filament، Spatie React، Permission، التتبع

Contents

Remerciements	i
Dedicace	ii
Dedicace	iii
Resume	iv
Abstract	v
المُلَخَّص	vi
List of Abbreviations	xiii
Introduction Generale	1
1 Generalities on the Web and Web Technologies	3
1.1 Presentation of the Web	4
1.1.1 Internet, Intranet, Extranet	4
1.1.2 Static and Dynamic Web Pages	4
1.1.3 HTTP/HTTPS Protocols and Client-Server Model	5
1.2 Frontend Technologies	5
1.2.1 React 19 and Component-Based Architecture	5

1.2.2	TypeScript for Type Safety	5
1.2.3	Tailwind CSS 4 — Utility-First Styling	6
1.2.4	Inertia.js — Server-Driven SPA Routing	6
1.2.5	Vite 8 — Modern Build Tooling	6
1.3	Backend Technologies	6
1.3.1	Laravel 13 Framework and Eloquent ORM	6
1.3.2	Filament 5 — Admin Panel Builder	7
1.3.3	PostgreSQL and SQLite Databases	7
1.4	Authentication and Authorization	7
1.4.1	Laravel Fortify — Authentication Scaffolding	7
1.4.2	Spatie Laravel Permission — Role-Based Access Control	8
2	Preliminary Study	10
2.1	Presentation of the Host Organization	11
2.2	Problem Statement	12
2.3	Objectives	12
2.4	Analysis of the Existing System	13
2.5	Critique of the Existing System	13
2.6	Feasibility Study	15
2.6.1	Technical Feasibility	15
2.6.2	Economic Feasibility	15
2.6.3	Operational Feasibility	16
2.6.4	Project Planning	16
3	Analysis and Design	18
3.1	UML Formalism	19
3.2	Actors	19
3.2.1	Codification	20
3.2.2	Rules of Management (Règles de Gestion)	21
3.3	Use Case Descriptions	22
3.3.1	Use Case 1: Authentication with 2FA and Passkeys	22
3.3.2	Use Case 2: Manage Organization Structure	23
3.3.3	Use Case 3: Create and Manage Employees	24
3.3.4	Use Case 4: Create Project with Requirements	25
3.3.5	Use Case 5: Create Assignment with Status Workflow	26
3.3.6	Use Case 6: View Dashboard Analytics	27
3.3.7	Use Case 7: Audit Trail	28
3.3.8	Use Case 8: Run Recommendation for Staffing	29
3.3.9	Use Case 9: Submit and Approve Profile Change Requests	30
3.4	Dynamic Study — Sequence Diagrams	31

3.5	Class Diagram	31
3.5.1	Class Descriptions	37
3.5.2	Passage to the Relational Model	39
3.6	Database Schema	41
3.6.1	Table: users	41
3.6.2	Table: employees	42
3.6.3	Table: assignments	42
3.6.4	Table: assignment_members	43
3.6.5	Table: audit_logs	43
3.6.6	Table: projects	44
3.6.7	Table: profile_change_requests	44
3.6.8	Table: recommendation_runs	45
3.6.9	Complete inventory of the database	45
3.7	Deployment Diagram	47
4	Implementation and Validation	48
4.1	Languages and Tools	49
4.1.1	PHP 8.3	49
4.1.2	Laravel 13	50
4.1.3	Filament 5	50
4.1.4	React 19	50
4.1.5	TypeScript	51
4.1.6	Inertia.js 3	51
4.1.7	Tailwind CSS 4	51
4.1.8	Vite 8	52
4.1.9	PostgreSQL	52
4.1.10	PHPUnit and Pest	52
4.1.11	Supporting Libraries and Tools	53
4.2	Application Structure	53
4.2.1	Architecture and Request Flow	53
4.2.2	Database Implementation	55
4.3	Interfaces	56
4.3.1	Filament Admin Dashboard	57
4.3.2	Employee Management Interface	58
4.3.3	Assignment Management Interface	59
4.3.4	Settings Pages	59
4.3.5	Login and Authentication Pages	60
4.3.6	Employee Self-Service Portal	61
4.3.7	Profile Change Request	61

4.3.8	Recommendation Runs	62
4.4	Testing and Validation	63
4.4.1	Test Architecture	63
4.4.2	Test Results	64
4.4.3	API Testing	65
4.4.4	Performance Testing	66
4.4.5	Responsive and Mobile Testing	67
	General Conclusion	68
4.5	Limitations	68
4.6	Future Perspectives	68
	References Bibliographiques	70
	Annexes	71

List of Figures

1.1	Web Architecture Overview — Client-Server Model	5
1.2	Inertia.js Bridge — Laravel to React Data Flow	6
1.3	Laravel MVC Request Lifecycle	8
1.4	RBAC Permission Model — Role-Permission Matrix	9
2.1	Organization structure of Djezzy (case-study areas)	11
2.2	Current manual resource assignment process	14
2.3	Planning of the project phases (weeks)	16
3.1	Use Case Diagram — System Actors and Use Cases	20
3.2	Activity diagram — Authentication with 2FA and passkeys	22
3.3	Activity diagram — Manage organization structure	23
3.4	Activity diagram — Create and manage employees	24
3.5	Activity diagram — Create project with requirements	25
3.6	Activity diagram — Assignment status workflow	27
3.7	Activity diagram — View dashboard analytics	28
3.8	Activity diagram — Audit trail	29
3.9	Activity diagram — Run recommendation for staffing	30
3.10	Activity diagram — Submit and approve profile change requests	31
3.11	Sequence diagram — Authentication with 2FA/passkeys	32

3.12	Sequence diagram — Manage organization structure	32
3.13	Sequence diagram — Create employee	33
3.14	Sequence diagram — Create project with requirements	33
3.15	Sequence diagram — Assignment status workflow	34
3.16	Sequence diagram — Dashboard analytics	34
3.17	Sequence diagram — Audit trail	35
3.18	Sequence diagram — Profile change request approval	35
3.19	Sequence diagram — Recommendation run	36
3.20	Class Diagram — Domain Model	36
3.21	Deployment Diagram — Server Infrastructure	47
4.1	PHP — Server-Side Scripting Language	50
4.2	Laravel — The PHP Framework for Web Artisans	50
4.3	Filament — Laravel Admin Panel Builder	50
4.4	React — User Interface Library	51
4.5	TypeScript — Typed Superset of JavaScript	51
4.6	Inertia.js — Server-Driven SPA Bridge for Laravel	51
4.7	Tailwind CSS — Utility-First CSS Framework	52
4.8	Vite — Frontend Build Tool	52
4.9	PostgreSQL — Relational Database Management System	52
4.10	PHPUnit and Pest — Testing Frameworks	53
4.11	Application Directory Structure	54
4.12	Filament Admin Dashboard — Stats and Charts	58
4.13	Employee Management — Profile with Relation Managers	58
4.14	Assignment Management — Status Workflow and Members	59
4.15	Settings Pages — Security with 2FA and Passkeys	60
4.16	Login Page — Two-Factor Challenge and Passkey Authentication	60
4.17	Employee Self-Service Portal — Profile, Workload and Assignments	61
4.18	Profile Change Request — Submission and Approval Workflow	62
4.19	Recommendation Runs — Ranked Candidates with Scores	63
4.20	Test Execution Output — All 68 Tests Passing	65

List of Tables

1.1	Role Definitions in the Djezzy Platform	8
3.1	System Actors	20

3.2	Codification of the business entities	21
3.3	Principal rules of management (cardinalities and constraints)	21
3.4	Use Case: Authentication with 2FA and Passkeys	22
3.5	Use Case: Manage Organization Structure	23
3.6	Use Case: Create and Manage Employees	24
3.7	Use Case: Create Project with Requirements	25
3.8	Use Case: Create Assignment with Status Workflow	26
3.9	Use Case: View Dashboard Analytics	27
3.10	Use Case: Audit Trail	28
3.11	Use Case: Run Recommendation for Staffing	29
3.12	Use Case: Submit and Approve Profile Change Requests	30
3.13	Class: User	37
3.14	Class: Employee	37
3.15	Class: Department	37
3.16	Class: Project	38
3.17	Class: Assignment	38
3.18	Class: Skill	38
3.19	Class: Certification	38
3.20	Class: Language	39
3.21	Class: AuditLog	39
3.22	Class: AssignmentStatusHistory	39
3.23	Mapping — UML classes to relational tables	40
3.24	Pivot tables created from the N-M associations	41
3.25	Table: users	41
3.26	Table: employees	42
3.27	Table: assignments	42
3.28	Table: assignment_members	43
3.29	Table: audit_logs	43
3.30	Table: projects	44
3.31	Table: profile_change_requests	44
3.32	Table: recommendation_runs	45
3.33	Inventory — Authentication, security and RBAC group	45
3.34	Inventory — Organization and employees group	46
3.35	Inventory — Catalog, projects and assignments group	46
3.36	Inventory — Portal, audit and recommendations group	46
4.1	Development Languages and Tools — Summary	49
4.2	User Interfaces	57
4.3	Test Results Summary	64

4.4	API Testing — Principal Authenticated Endpoints	66
4.5	Performance Testing — Average Response Times (local environment)	66
4.6	Responsive Testing — Viewport Validation Results	67

List of Abbreviations

Abbreviation	Signification
ACID	Atomicity, Consistency, Isolation, Durability (database transaction properties)
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
FK	Foreign Key
HMR	Hot Module Replacement (instant reload of modified modules)
HTML	HyperText Markup Language
HTTP / HTTPS	HyperText Transfer Protocol / Secure
JS	JavaScript
LEMP	Linux, Nginx, MySQL/PostgreSQL, PHP (server stack)
MVC	Model-View-Controller (architectural pattern)
ORM	Object-Relational Mapping
PHP	Hypertext Preprocessor (scripting language)
PK	Primary Key
RBAC	Role-Based Access Control
SPA	Single-Page Application
SQL	Structured Query Language
TLS / SSL	Transport Layer Security / Secure Sockets Layer
UML	Unified Modeling Language
URL	Uniform Resource Locator

Introduction Generale

1. General Context

Djezzy (Optimum Telecom Algerie), the leading mobile operator in Algeria with more than 18 million subscribers, operates business units whose performance depends on the availability of qualified employees at the right time. The assignment of scarce human resources to projects, departments, and teams is currently handled manually, relying on spreadsheets, e-mail exchanges, and the experience of department heads. This manual approach slows down the preparation of proposals, creates fairness and transparency problems across business units, and provides no reliable way to check, before a commitment is made, whether the required profiles (skills, certifications, languages, availability, current workload) are actually available. The present work, carried out within the Information Systems department of Djezzy, aims to replace this manual practice by a centralized resource assignment platform.

2. Problem Statement and Working Hypotheses

Problem statement. How can an organization such as Djezzy ensure that every new project assignment is built on objective, up-to-date information, while respecting the confidentiality levels of the projects and the workloads of the employees? The manual process raises several concrete questions:

- How to obtain, in real time, an inventory of employees whose profile (skills, certifications, languages) matches the requirements of a project?
- How to guarantee that an employee is available (current workload, leave, assignment dates) before being proposed to a business unit?
- How to keep a complete and auditable history of every assignment decision?

Working hypotheses. We assume that (i) a centralized database consolidating employee profiles, workloads, and project requirements makes it possible to answer these questions automatically; (ii) a role-based authorization model (RBAC) allows each actor (HR administrator, project coordinator, department head, employee) to act only within his or her prerogatives; and (iii) a recommendation engine based on a multi-criteria scoring function produces relevant candidate shortlists.

3. Contribution

The contribution of this work is a **Resource Assignment Platform** for Djezzy, whose main objectives are:

1. **Centralize** all reference data: employees, departments, teams, positions, skills, certifications, languages, projects, and assignments;
2. **Automate** the search for candidate employees by matching project requirements against employee profiles (skill-, certification-, and language-based criteria);
3. **Score and rank** the candidates with a weighted recommendation function incorporating proficiency levels, workload, and assignment history;
4. **Manage** the full assignment lifecycle (proposal, approval, acceptance, rejection, feedback) with a controlled status machine;
5. **Guarantee** traceability by an audit log recording every sensitive operation (who, what, when, from which IP);
6. **Protect** the application with a role-based access control mechanism covering six roles and fifty-two permissions.

4. Report Organization

The report is organized as follows. **Chapter I** recalls the generalities on the Web and the technologies used: HTTP, frontend (React, TypeScript, Tailwind CSS, Inertia), backend (Laravel, Filament, PostgreSQL), and security mechanisms (Fortify, Spatie Permission). **Chapter II** presents the preliminary study: the host organization (Djezzy), the analysis of the existing process, and the critique that justifies our project. **Chapter III** details the analysis and design: UML formalism, use cases, activity and sequence diagrams, class diagram, codification rules, business rules, the passage to the relational model, and the deployment diagram. **Chapter IV** describes the realization and validation: technologies, application structure, screenshots of the realized interfaces, and the tests performed (unit, feature, API, performance, responsive). We finish with the general conclusion and perspectives.

CHAPTER I

Generalities on the Web and Web Technologies

Introduction

The development of modern web applications requires a solid understanding of the technologies that power today's Internet. From the client-server architecture to single-page applications (SPAs) and server-side rendered frameworks, the web ecosystem has evolved dramatically over the past two decades. This chapter provides a comprehensive overview of the web technologies underpinning the Djezzy Resource Assignment Platform, covering frontend frameworks, backend architectures, database management, and security mechanisms.

1.1 Presentation of the Web

The **World Wide Web** (WWW) is a system of interlinked hypertextual documents accessed via the Internet using protocols such as HTTP (Hypertext Transfer Protocol) and its secure variant HTTPS. Invented by Tim Berners-Lee in 1989 at CERN, the web has evolved through several generations:

- **Web 1.0** (Static Web): Read-only HTML pages with minimal interactivity. Content was authored by website owners and consumed passively by users.
- **Web 2.0** (Social Web): Interactive platforms with user-generated content, social networking, and rich media. Technologies like AJAX enabled dynamic page updates without full reloads.
- **Web 3.0** (Semantic Web): Emerging standards for machine-readable data, decentralized architectures, and intelligent web services powered by artificial intelligence.

1.1.1 Internet, Intranet, Extranet

The **Internet** is the global network of networks connecting billions of devices. An **Intranet** is a private network restricted to an organization's internal users, while an **Extranet** extends limited access to trusted external partners. The Djezzy platform is designed as an Intranet application deployed within the company's infrastructure, accessible to authorized employees through authenticated sessions.

1.1.2 Static and Dynamic Web Pages

Static pages are delivered as-is from the server, containing fixed HTML content. **Dynamic pages** are generated on-the-fly by server-side scripts (PHP, Python, Node.js) that query databases and apply business logic before producing the final HTML. The Djezzy platform generates all content dynamically, as every page requires real-time data from the PostgreSQL database.

1.1.3 HTTP/HTTPS Protocols and Client-Server Model

The client-server architecture is the foundational model of web applications. A **client** (typically a web browser) sends HTTP requests to a **server**, which processes the request and returns an HTTP response containing HTML, CSS, JavaScript, or data (JSON). HTTPS adds TLS/SSL encryption to protect data in transit. The Djezzy platform uses HTTPS for all communications to ensure the confidentiality and integrity of sensitive HR and project data.

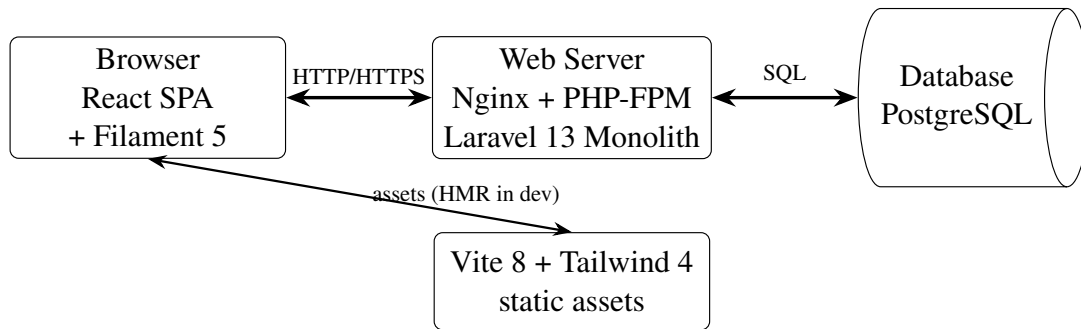


Fig. 1.1: Web Architecture Overview — Client-Server Model

1.2 Frontend Technologies

Modern frontend development relies on component-based JavaScript frameworks that enable the creation of rich, interactive user interfaces. The Djezzy platform leverages a modern frontend stack composed of React, TypeScript, Tailwind CSS, and Inertia.js.

1.2.1 React 19 and Component-Based Architecture

React is a declarative, component-based JavaScript library developed by Meta (formerly Facebook). React 19 introduces enhanced server components, improved concurrent rendering, and better integration with server-side frameworks. Each UI element is encapsulated in a self-contained component that manages its own state and rendering logic. This architecture promotes code reuse, testability, and maintainability—critical qualities for a complex enterprise application like the Djezzy platform.

1.2.2 TypeScript for Type Safety

TypeScript is a statically typed superset of JavaScript that compiles to plain JavaScript. It introduces interfaces, enums, generics, and type annotations that catch errors at compile time rather than runtime. In a large-scale application with numerous data models (employees, projects, assignments), TypeScript significantly reduces the risk of type-related bugs and improves developer productivity through autocompletion and refactoring support.

1.2.3 Tailwind CSS 4 — Utility-First Styling

Tailwind CSS is a utility-first CSS framework that provides atomic classes for spacing, typography, colors, and layout. Instead of writing custom CSS, developers compose UI styles by combining utility classes directly in HTML/JSEX markup. Tailwind CSS 4 introduces improved performance through JIT (Just-In-Time) compilation and enhanced dark mode support, both of which are used in the Djezzy platform’s admin panel and user-facing interfaces.

1.2.4 Inertia.js — Server-Driven SPA Routing

Inertia.js is a protocol that bridges server-side routing (Laravel) with client-side rendering (React) without requiring a separate API layer. When a user clicks a link, Inertia intercepts the request, makes an XHR call to the Laravel backend, receives a JSON response containing the component name and props, and renders the corresponding React component on the client side. This approach preserves the simplicity of traditional server-side routing while delivering the fluid user experience of a single-page application.

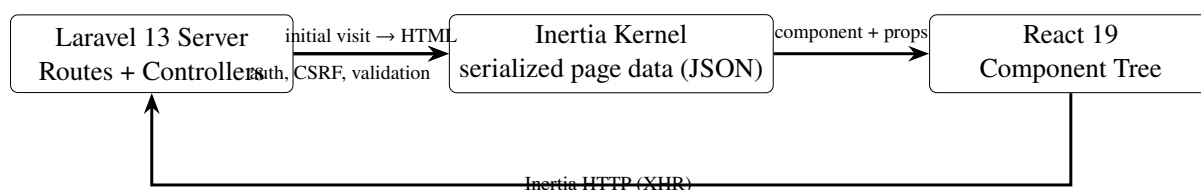


Fig. 1.2: Inertia.js Bridge — Laravel to React Data Flow

1.2.5 Vite 8 — Modern Build Tooling

Vite is a next-generation frontend build tool that provides instantaneous server startup through native ES module imports and optimized production builds using Rollup. Vite 8 offers improved dependency pre-bundling, enhanced plugin compatibility, and faster hot module replacement (HMR), making it the preferred choice for developing the React/Inertia frontend of the Djezzy platform.

1.3 Backend Technologies

The backend of the Djezzy platform is built on Laravel, a mature PHP MVC framework, extended with Filament for the admin panel interface.

1.3.1 Laravel 13 Framework and Eloquent ORM

Laravel 13 is the latest major release of the most popular PHP framework. It provides a comprehensive toolkit including:

- **Eloquent ORM:** An active-record implementation for database operations with relationships, scopes, and eager loading
- **Migrations:** Version-controlled database schema definitions
- **Middleware:** Request/response processing pipeline for authentication, authorization, and logging
- **Service Container:** Dependency injection container for managing class dependencies
- **Queue System:** Background job processing for asynchronous tasks
- **Event System:** Observer pattern implementation for decoupled application logic

1.3.2 Filament 5 — Admin Panel Builder

Filament is an open-source admin panel builder for Laravel that dramatically accelerates the development of CRUD interfaces. Filament 5 provides:

- **Resources:** Automatically generated list, create, edit, and view pages for Eloquent models
- **Relation Managers:** Inline management of related models (e.g., employee skills, project requirements)
- **Widgets:** Dashboard components including stat cards, charts, and data tables
- **Forms and Tables:** Declarative API for building complex forms and sortable, filterable data tables
- **Authorization:** Integration with Spatie Permission for role-based resource access

1.3.3 PostgreSQL and SQLite Databases

PostgreSQL is the recommended relational database for production deployment, offering ACID compliance, advanced indexing, partial unique indexes (used for the single-active-assignment constraint), JSON column support, and robust concurrent access handling. **SQLite** is used as a lightweight alternative for local development and automated testing, providing fast setup and tear-down for test suites.

1.4 Authentication and Authorization

Security is paramount for a platform handling sensitive employee and organizational data. The Djazzy platform implements multiple layers of authentication and authorization.

1.4.1 Laravel Fortify — Authentication Scaffolding

Laravel Fortify provides a backend implementation of authentication features including:

- Registration and login with email/password
- Email verification and password reset flows

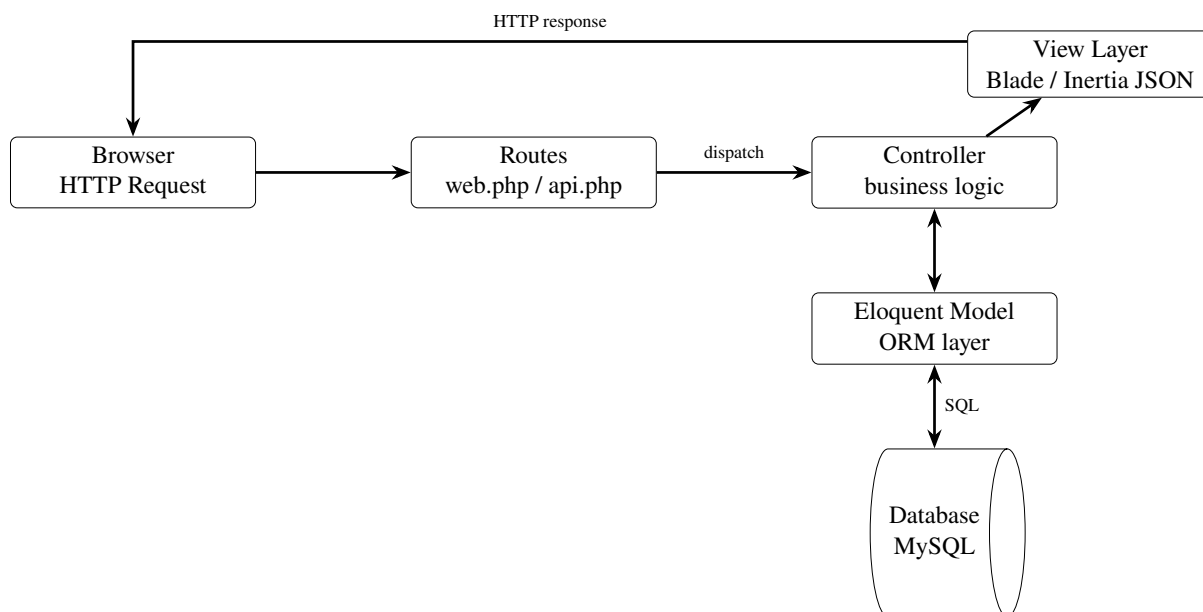


Fig. 1.3: Laravel MVC Request Lifecycle

- **Two-Factor Authentication (2FA):** Time-based One-Time Password (TOTP) via authenticator apps, with encrypted secret storage and recovery codes
- **Passkeys (WebAuthn):** Passwordless authentication using public-key cryptography, supporting biometric verification and hardware security keys
- Profile management and account deletion

1.4.2 Spatie Laravel Permission — Role-Based Access Control

The Djizzy platform implements a comprehensive RBAC model using Spatie Laravel Permission. Six roles are defined with escalating privilege levels:

Tab. 1.1: Role Definitions in the Djizzy Platform

Role	Description
Super Admin	Full system access: manages users, roles, permissions, and all organizational entities
Admin	Manages organizational structure, employees, projects, and assignments with broad permissions
HR Manager	Manages employee profiles, skills, certifications, languages, availability, and workload tracking
Resource Manager	Creates and manages assignments, tracks workload distribution, and monitors availability
Project Manager	Creates projects, defines requirements, submits projects for staffing, and tracks project status
Employee	Views assigned projects and updates personal profile information

A total of 52 granular permissions are distributed across 14 groups (Business Units, De-

partments, Teams, Positions, Locations, Employees, Skills, Certifications, Languages, Projects, Assignments, Recommendations, Users, and Roles), each with view/create/update/delete operations. This fine-grained model ensures that users can only access the data and operations relevant to their role.

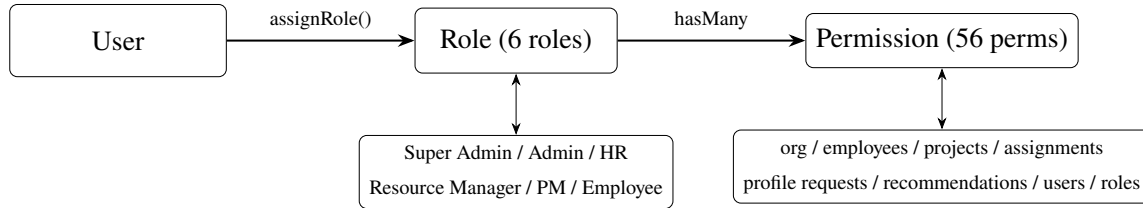


Fig. 1.4: RBAC Permission Model — Role-Permission Matrix

Conclusion

This chapter has presented the theoretical foundations and technological landscape underpinning the Djezzzy Resource Assignment Platform. The web has evolved from static document repositories to dynamic, application-rich environments powered by sophisticated client-server architectures. The frontend stack (React, TypeScript, Tailwind CSS, Inertia.js, Vite) delivers a modern single-page application experience, while the backend stack (Laravel 13, Filament 5, PostgreSQL) provides a robust, secure, and maintainable server-side foundation. The authentication and authorization mechanisms — featuring 2FA, passkeys, and fine-grained RBAC — ensure that the platform meets enterprise-grade security requirements. These technologies form the technical backbone upon which the Djezzzy platform is built, as detailed in the subsequent chapters.

CHAPTER II

Preliminary Study

Introduction

The preliminary study phase is essential for understanding the context, constraints, and requirements of the project. This chapter presents the host organization Djazzy, analyzes the problem that motivated this project, defines the objectives and scope of the proposed solution, and examines the limitations of the existing resource allocation system.

2.1 Presentation of the Host Organization

Djazzy (Orascom Telecom Algeria) is one of Algeria's leading telecommunications operators, providing mobile, internet, and enterprise services to millions of subscribers across the national territory. As a major player in the Algerian telecom market, Djazzy employs thousands of professionals organized across multiple business units, departments, and project teams.

The **IT** and **HR** departments jointly manage employee assignments across projects, requiring tight coordination between:

- **Organizational structure:** Business units, departments, teams, positions, and locations forming a multi-level hierarchy
- **Employee profiles:** Skills, certifications, languages, availability status, and workload data for each employee
- **Project requirements:** Typed requirements specifying the qualifications needed for each project

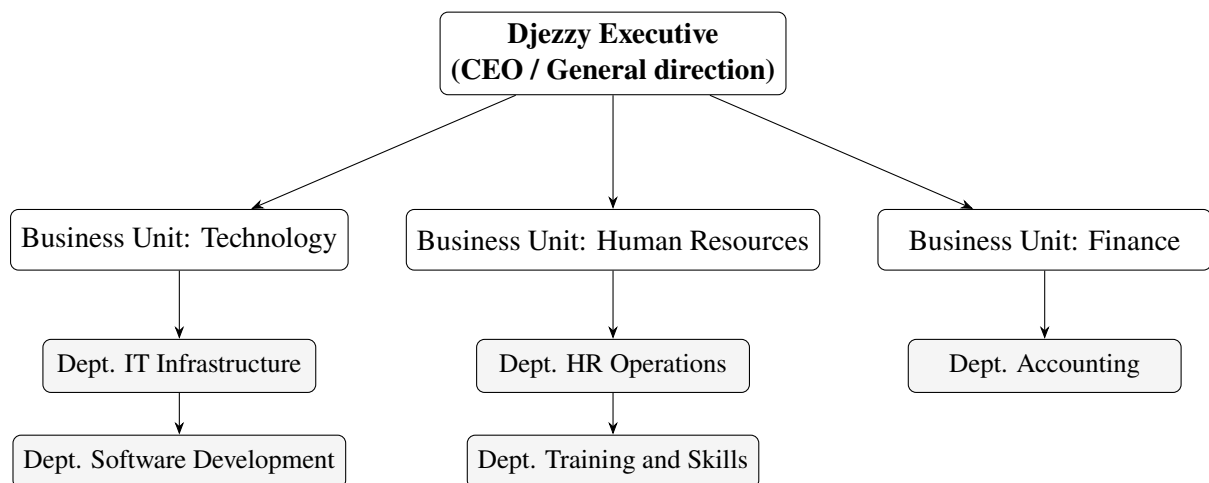


Fig. 2.1: Organization structure of Djazzy (case-study areas)

Resource allocation is currently performed manually using spreadsheets and email communication between HR managers and project managers, leading to inefficiencies, data inconsistencies, and a lack of traceability in assignment decisions.

2.2 Problem Statement

The central question driving this project is:

How to centralize and streamline the resource assignment process across IT and non-IT projects while maintaining full audit traceability, enforcing business rules, and providing real-time workload visibility?

More specifically, the problem can be decomposed into the following sub-questions:

- How to centralize organizational structure management (business units, departments, teams, positions, locations) in a unified system?
- How to build comprehensive employee profiles with verifiable skills, certifications, languages, availability status, and workload tracking?
- How to implement a project requirement system with typed requirements (skill, certification, language, experience, position, department, location, availability)?
- How to design an assignment workflow with a clear status machine (Pending → Approved → Active → Completed) plus Rejected and Cancelled states?
- How to enforce a single active assignment per project while supporting multiple assignment modes (single employee, multiple employees, team, department)?
- How to maintain a complete audit trail of assignment decisions with actor identification, timestamps, old/new values, and client information?

2.3 Objectives

Based on the problem analysis, the following objectives have been defined for the Djazzy Resource Assignment Platform:

1. **Centralize organizational structure management:** Provide a unified interface for managing business units, departments, teams, positions, and locations with hierarchical relationships and full CRUD operations.
2. **Build comprehensive employee profiles:** Create employee records linked to organizational entities, enriched with skills (with proficiency levels), certifications (with verification status), languages (with proficiency levels), availability status, and workload percentages.
3. **Implement a typed requirement system:** Allow project managers to define project requirements across multiple types (skill, certification, language, experience, position, department, location, availability) with minimum levels and weights.
4. **Design an assignment workflow:** Implement a state-machine-driven assignment lifecycle with status transitions, approval workflows, and the enforcement of a single-active-assignment-per-project rule via database-level constraints.
5. **Provide full audit trail:** Record every assignment action (create, approve, activate, com-

plete, reject, cancel) with actor ID, action identifier, old/new values, IP address, and user agent string.

6. **Build an admin dashboard:** Deliver statistical widgets, workload charts, alert tables, and recent activity logs for real-time organizational visibility.
7. **Implement role-based access control:** Enforce a 6-role, 52-permission RBAC model using Spatie Laravel Permission, ensuring that users can only access data and operations relevant to their role.
8. **Secure authentication:** Integrate two-factor authentication (TOTP) and passkey (WebAuthn) support via Laravel Fortify for enterprise-grade security.

2.4 Analysis of the Existing System

The current resource allocation process at Djezzy relies on the following manual workflows:

1. **Employee tracking:** HR managers maintain employee profiles in Excel spreadsheets, recording basic information, skills, and certifications. These spreadsheets are shared via email and are not version-controlled.
2. **Project definition:** Project managers create project descriptions in documents or spreadsheets, specifying requirements informally during meetings.
3. **Assignment process:** Resource managers identify suitable employees by manually cross-referencing skill spreadsheets with project requirements, then communicate assignments via email.
4. **Workload estimation:** Workload is estimated verbally during weekly meetings, with no quantitative tracking or automated calculation.
5. **Status tracking:** Project and assignment statuses are updated manually in shared documents, with no automated state transitions or validation rules.

2.5 Critique of the Existing System

The analysis of the current system reveals several critical deficiencies:

1. **No centralized employee profile:** Employee skills and certifications are tracked informally across multiple spreadsheets with no single source of truth. There is no mechanism to verify the accuracy or currency of skill data.
2. **Subjective workload estimation:** Workload is estimated verbally without quantitative tracking, leading to inaccurate assessments and potential over-allocation of resources.
3. **No audit trail:** Assignment decisions and status changes are not recorded, making it impossible to trace who made a decision, when it was made, and what the previous state was.
4. **Undetected over-allocation risk:** Multiple managers can claim the same employee simultaneously because there is no enforcement of single-active-assignment rules. Over-allocation

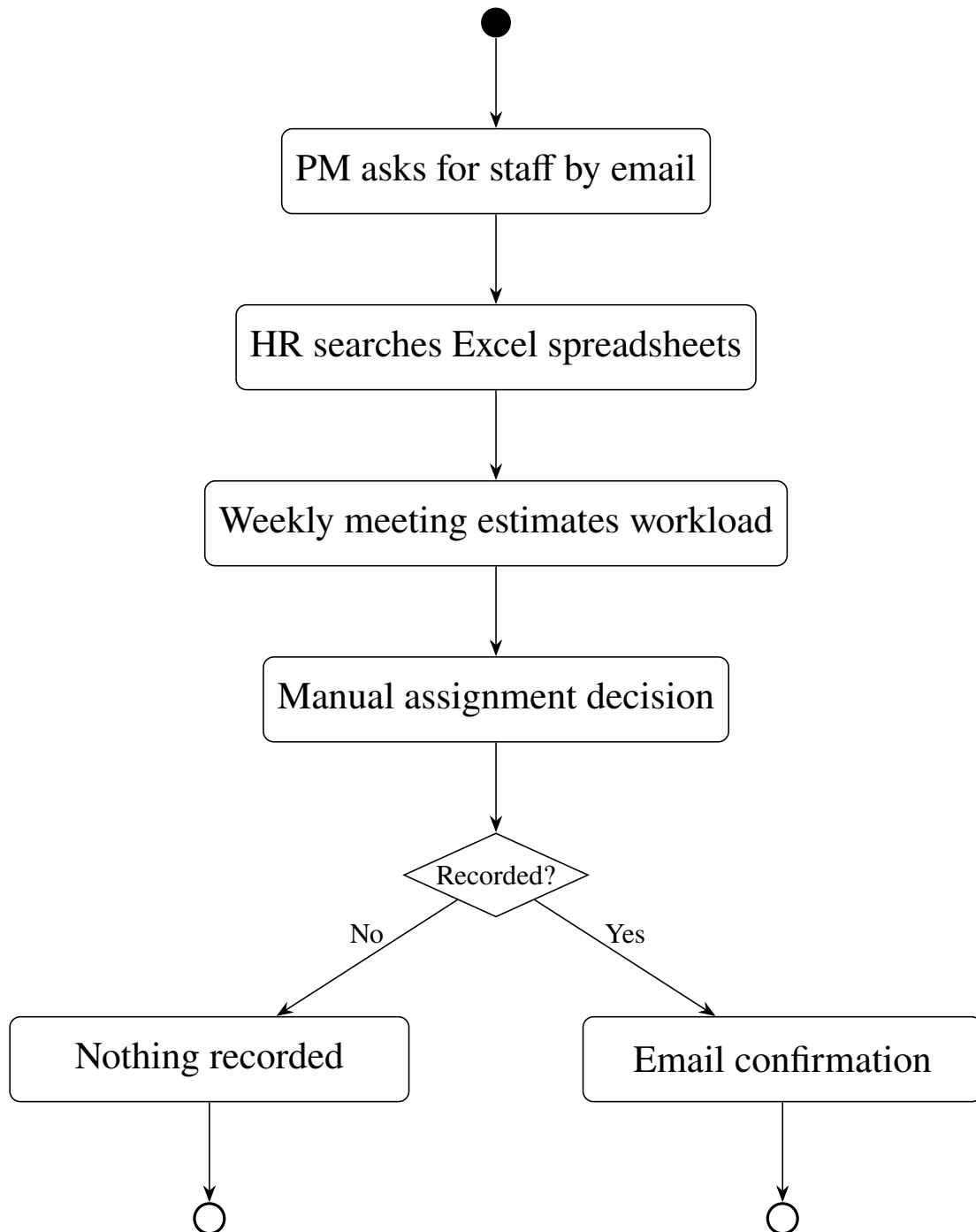


Fig. 2.2: Current manual resource assignment process

goes undetected until performance degrades.

5. **No assignment history:** When spreadsheets are overwritten or deleted, assignment history is permanently lost, preventing analysis of past resource utilization.
6. **Manual error-prone processes:** Email-based communication and manual data entry introduce frequent errors, delays, and miscommunication between HR, project managers, and resource managers.

These deficiencies highlight the urgent need for a centralized, automated, and auditable resource assignment platform that addresses all identified limitations.

2.6 Feasibility Study

A feasibility study was conducted before launching the development to verify that the proposed platform is technically, economically, operationally, and organizationally viable within the constraints of Djezzy.

2.6.1 Technical Feasibility

The chosen stack is mature, widely adopted, and well documented, which considerably reduces the technical risk. The server side relies on **Laravel 13** and **PHP 8.3**, two technologies with a large ecosystem and strong community support. The administration interface is built with **Filament 5**, while the employee self-service portal uses **React 19** delivered through the **Inertia.js** bridge. The relational database management system is **PostgreSQL** (compatible with MySQL). The whole platform runs on a standard **Nginx + PHP-FPM** server already available in Djezzy's infrastructure. No specialized hardware is required, and every actor accesses the platform through a standard web browser. The required skills (PHP, Laravel, SQL, modern front-end development) are present within the development team, so no external training or costly consultants are needed.

2.6.2 Economic Feasibility

The project uses exclusively **free open-source technologies** (PHP, Laravel, React, Tailwind CSS, PostgreSQL). No software licenses must be acquired. The development is carried out internally, and the deployment reuses the existing hosting infrastructure. The main cost is therefore the working time of the team. The expected gains largely offset this cost: suppression of manual spreadsheet maintenance, reduction of allocation errors, real-time workload visibility, and a complete audit trail that satisfies internal compliance requirements. The platform is therefore economically justified.

2.6.3 Operational Feasibility

Operationally, the platform is adopted progressively. The **role-based access control** model lets each department keep its own scope of work: administrators and HR managers manage the referential data, resource managers handle assignments, and employees use the self-service portal to consult their assignments and submit profile change requests. A short training session is planned for each profile, and the interface has been designed to be taken in hand by nontechnical users. The validation campaign (described in Chapter IV) guarantees the reliability of the system before its production launch.

2.6.4 Project Planning

In order to frame the development activities, the project was organized into five phases: preliminary study, analysis and design, implementation, testing and validation, and documentation and delivery. Figure 2.3 presents the overall planning over a period of twenty-two weeks.

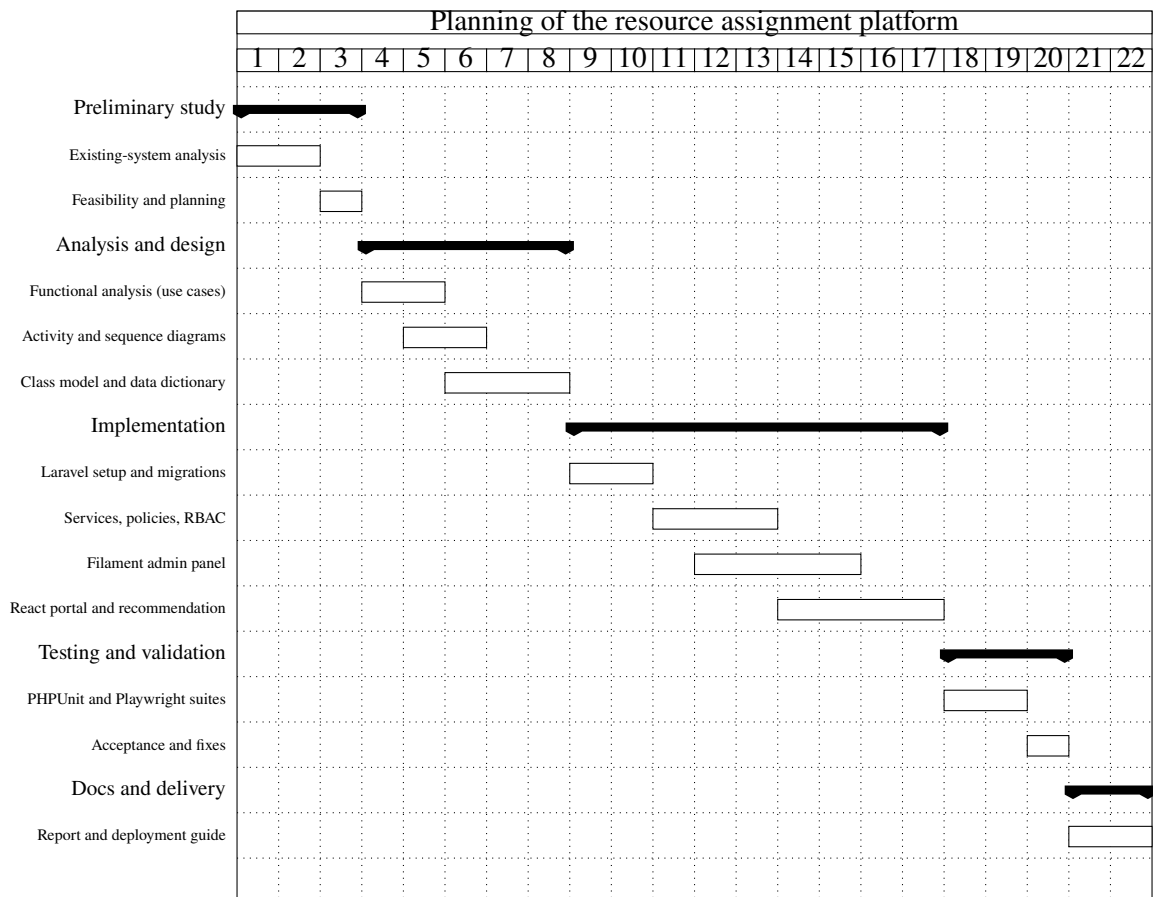


Fig. 2.3: Planning of the project phases (weeks)

Conclusion

This preliminary study has provided a thorough understanding of the Djazzy organization, the problems associated with the current manual resource allocation process, and the specific objectives of the proposed platform. The critique of the existing system confirms that manual spreadsheet-based workflows are inadequate for managing the complexity of resource assignments in a large telecommunications company. The next chapter presents the analysis and design phase, including use case modeling, UML diagrams, database schema design, and deployment architecture.

CHAPTER III

Analysis and Design

Introduction

This chapter presents the analysis and design of the Djazzy Resource Assignment Platform using the Unified Modeling Language (UML) formalism. It identifies the system actors, describes the use cases and their flows, presents the class diagram with attribute and method specifications, defines the database schema, and outlines the deployment architecture. The design phase bridges the gap between the requirements gathered during the preliminary study and the implementation detailed in the next chapter.

3.1 UML Formalism

The **Unified Modeling Language** (UML) is a standardized visual modeling language used to specify, visualize, construct, and document the artifacts of software systems. UML provides various diagram types for different perspectives:

- **Use case diagrams:** Capture functional requirements from the user's perspective
- **Sequence diagrams:** Model the interaction between objects over time for a specific scenario
- **Class diagrams:** Define the static structure of the system (classes, attributes, methods, relationships)
- **Deployment diagrams:** Describe the physical deployment of the system on hardware infrastructure

In this project, UML is used to model the six actors, nine use cases, ten classes, and the deployment topology of the Djazzy platform.

3.2 Actors

The following actors have been identified through stakeholder analysis and requirements gathering:

Tab. 3.1: System Actors

Actor	Description
Super Admin	Full system access: manages users, roles, permissions, and all organizational entities
Admin	Manages organizational structure, employees, projects, and assignments with broad permissions
HR Manager	Manages employee profiles, skills, certifications, languages, availability, and workload tracking
Resource Manager	Creates and manages assignments, tracks workload distribution, and monitors availability
Project Manager	Creates projects, defines requirements, submits projects for staffing, and tracks project status
Employee	Views assigned projects and updates personal profile information

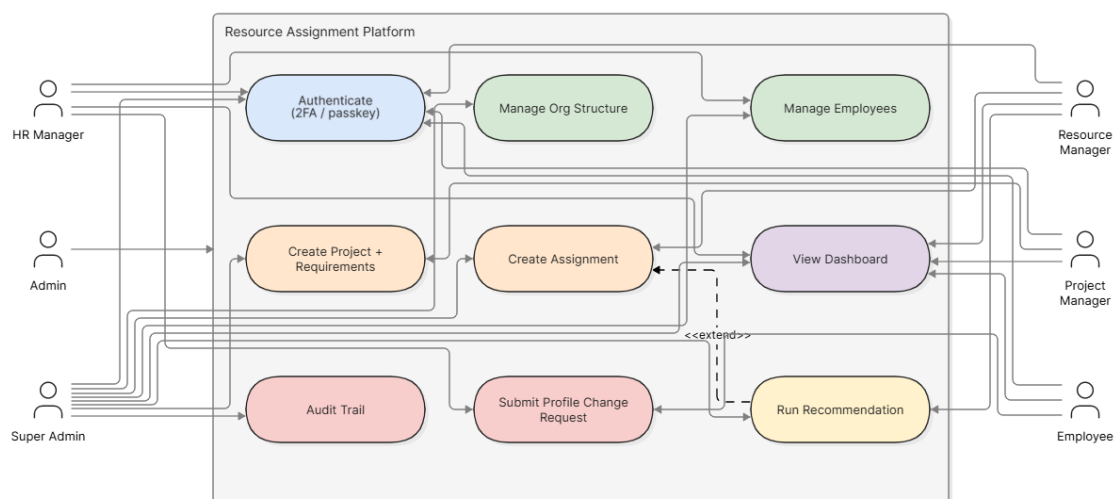


Fig. 3.1: Use Case Diagram — System Actors and Use Cases

3.2.1 Codification

In order to identify unambiguously and in a human-readable way the entities manipulated by the system, a systematic codification was defined. Each business entity receives a code following the format PREFIX-N[XX]-[YY], where PREFIX identifies the type of the entity, N[XX] is its sequential number within the type, and [YY] is the two-digit year of creation. The code is generated automatically when the entity is created and is guaranteed unique. Table 3.2 summarizes the codification rules applied by the platform.

Tab. 3.2: Codification of the business entities

Element	Signification	Example
Employee	EMPL-N [XX] - [YY]	EMPL-N03-25
Department	DEPT-N [XX] - [YY]	DEPT-N02-25
Team	TEAM-N [XX] - [YY]	TEAM-N04-25
Position	POS-N [XX] - [YY]	POS-N01-25
Location	LOC-N [XX] - [YY]	LOC-N03-25
Project	PROJ-N [XX] - [YY]	PROJ-N07-25
Skill	SKILL-N [XX] - [YY]	SKILL-N01-25
Certification	CERT-N [XX] - [YY]	CERT-N05-25
Assignment	ASG-N [XX] - [YY]	ASG-N12-25

3.2.2 Rules of Management (Règles de Gestion)

The rules of management express, in natural language, the business constraints that the system must enforce independently of any technical choice. Table 3.3 lists the principal rules applied by the platform; several of them are enforced at the database level in addition to the application layer.

Tab. 3.3: Principal rules of management (cardinalities and constraints)

Rule	Description
RG01	A User is linked to at most one Employee; the foreign key <code>employees.user_id</code> is protected by RESTRICT on delete.
RG02	A Business Unit contains one-to-many Departments; a Department contains one-to-many Teams; a Department has zero or one ParentDepartment.
RG03	An Employee belongs to one Department, one Team, one Position, one Location and zero or one Manager (self-reference).
RG04	A Skill, a Certification and a Language are linked many-to-many to Employees through pivot tables (<code>employee_skills</code> , <code>employee_certifications</code> , <code>employee_languages</code>) carrying the proficiency level (1–5).
RG05	An Employee has one-to-many Availability records and one-to-many Workload records; all percentages are constrained between 0 and 100.
RG06	A Project has one-to-many Requirements (typed); it links many-to-many to Skills, Certifications and Languages through pivots carrying a minimum level and a weight.
RG07	An Assignment concerns exactly one Project; it links many-to-many to Employees (<code>assignment_members</code>) and to Teams (<code>assignment_teams</code>).
RG08	At most one Approved or Active assignment exists per Project (enforced by a partial unique index).
RG09	Every status transition of an Assignment is recorded in <code>assignment_status_history</code> with the actor and the reason.
RG10	An Employee generates zero or more ProfileChangeRequests; a User (reviewer) validates zero or more of them before the changes are applied.
RG11	A RecommendationRun produces an ordered list of Recommendations (rank, score); a Project can receive several candidate evaluations.

3.3 Use Case Descriptions

3.3.1 Use Case 1: Authentication with 2FA and Passkeys

Tab. 3.4: Use Case: Authentication with 2FA and Passkeys

Field	Description
Title	Authentication with 2FA and Passkeys
Actors	All Users
Description	User logs in with email/password. If 2FA is enabled, a TOTP code or recovery code is required. Passkeys offer passwordless login via WebAuthn.
Pre-conditions	User has valid account. 2FA may be enabled. Passkey may be registered.
Post-conditions	User is authenticated and redirected to dashboard or admin panel based on role.
Main Flow	1. User navigates to login page and enters email and password 2. System validates credentials via Laravel Fortify 3. If 2FA is enabled, system displays TOTP challenge 4. User enters 6-digit code or recovery code 5. System verifies code and creates authenticated session
Alternative Flow	Passkey login: system triggers WebAuthn challenge, user verifies via biometric/security key, session is created without password.

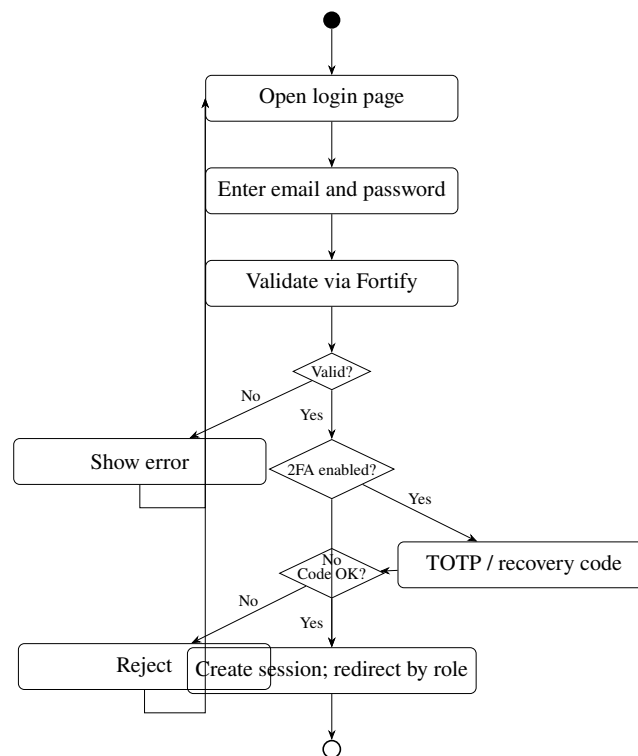


Fig. 3.2: Activity diagram — Authentication with 2FA and passkeys

3.3.2 Use Case 2: Manage Organization Structure

Tab. 3.5: Use Case: Manage Organization Structure

Field	Description
Title	Manage Organization Structure
Actors	Super Admin, Admin
Description	Admin creates and manages business units, departments, teams, positions, and locations with hierarchical relationships (departments belong to business units, teams belong to departments, positions belong to teams).
Pre-conditions	User authenticated with appropriate permissions.
Post-conditions	Organization structure is updated with new or modified entities.
Main Flow	1. Admin navigates to organization section in Filament panel 2. Admin selects entity type (business unit, department, team, position, or location) 3. System displays list view with search, filters, and sort 4. Admin creates or edits entity with form validation 5. System saves changes with audit log entry
Alternative Flow	Invalid parent reference triggers validation error. Deleting a parent with children triggers restrictOnDelete constraint.

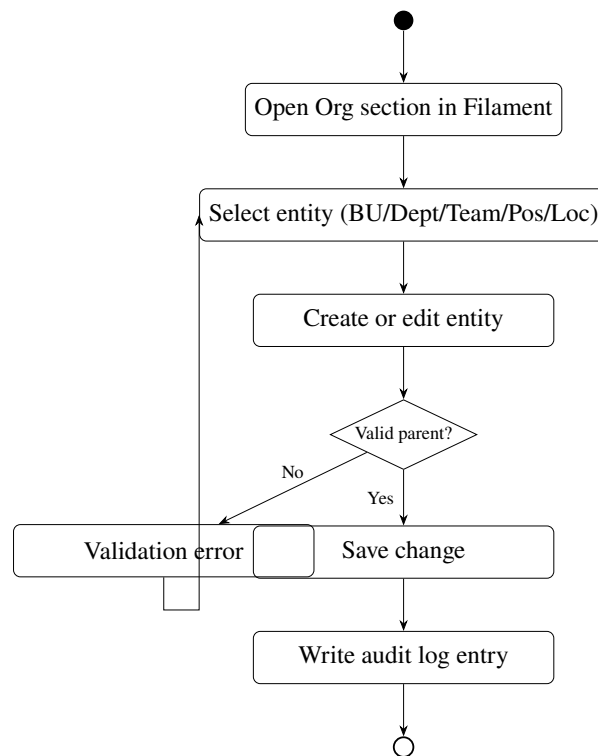


Fig. 3.3: Activity diagram — Manage organization structure

3.3.3 Use Case 3: Create and Manage Employees

Tab. 3.6: Use Case: Create and Manage Employees

Field	Description
Title	Create and Manage Employees
Actors	Super Admin, Admin, HR Manager
Description	HR creates employee profiles linked to organizational entities, then enriches with skills, certifications, languages, availability status, and workload data.
Pre-conditions	User authenticated with employee management permissions. Organizational structure exists.
Post-conditions	Employee profile is created with complete profile data.
Main Flow	1. HR navigates to employee management in Filament panel 2. HR creates new employee with personal info, department, position, location, employment type and status 3. HR adds skills with proficiency levels (beginner/intermediate/advanced/native) 4. HR adds certifications with verification status (pending/verified/rejected/expired) 5. HR adds languages with proficiency levels 6. HR sets availability status (available/partially available/unavailable/on leave) and workload percentages
Alternative Flow	Editing employee linked to active assignment triggers warning. Soft-deleting employee preserves assignment history.

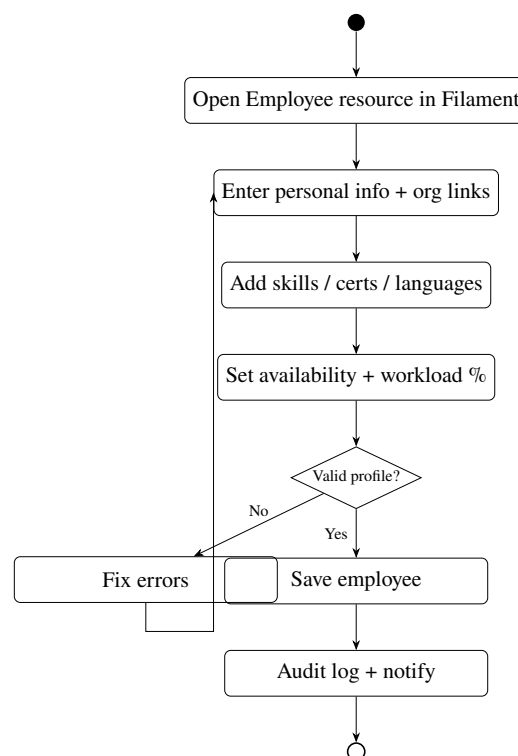


Fig. 3.4: Activity diagram — Create and manage employees

3.3.4 Use Case 4: Create Project with Requirements

Tab. 3.7: Use Case: Create Project with Requirements

Field	Description
Title	Create Project with Requirements
Actors	Super Admin, Admin, Project Manager
Description	Project manager creates a project, defines category, assignment mode, priority, dates, confidentiality level, and adds typed requirements (skills, certifications, languages, experience, position, department, location, availability).
Pre-conditions	User authenticated with project management permissions. Project categories exist.
Post-conditions	Project is created in Draft status with defined requirements.
Main Flow	1. PM navigates to project management in Filament panel 2. PM creates new project with name, description, category, assignment mode, priority, status, dates, and confidentiality level 3. System saves project in Draft status 4. PM adds project requirements (skill, certification, language, or other requirement types) with minimum level and weight 5. System validates and stores requirements
Alternative Flow	Changing assignment mode after assignment exists triggers validation. Status transitions follow: Draft → Submitted → Under Review → Staffing → Assigned → In Progress → Completed.

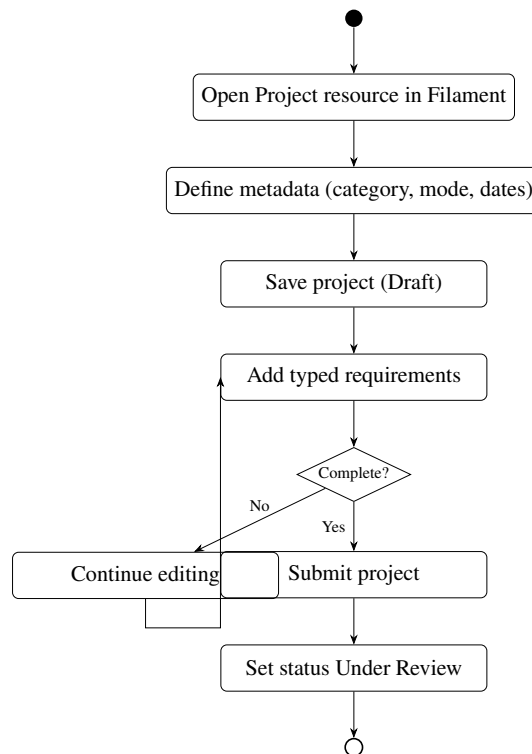


Fig. 3.5: Activity diagram — Create project with requirements

3.3.5 Use Case 5: Create Assignment with Status Workflow

Tab. 3.8: Use Case: Create Assignment with Status Workflow

Field	Description
Title	Create Assignment with Status Workflow
Actors	Super Admin, Admin, Resource Manager
Description	Resource manager creates an assignment linking employees or teams to a project, then progresses through status machine: Pending → Approved → Active → Completed (or Rejected/Cancelled at various stages).
Pre-conditions	User authenticated with assignment permissions. Project exists. Employees are active.
Post-conditions	Assignment is created and status history is recorded. Single-active-per-project rule is enforced.
Main Flow	1. RM navigates to assignment management in Filament panel 2. RM creates new assignment for a project with assignment type (employee/team/department) 3. System validates no current approved or active assignment exists for the project (partial unique index) 4. RM assigns members (employees) with allocation percentage, responsibility, and dates 5. RM assigns teams if applicable 6. Assignment starts in Pending status 7. Approver transitions assignment to Approved (records approver and timestamp) 8. RM activates the assignment (transitions to Active) 9. Upon completion, RM marks assignment Completed (triggers left_at date on all members)
Alternative Flow	Transition from Pending to Rejected or Cancelled records reason in status history. Cancellation allowed from Pending, Approved, or Active status only. DomainException thrown for invalid transitions.

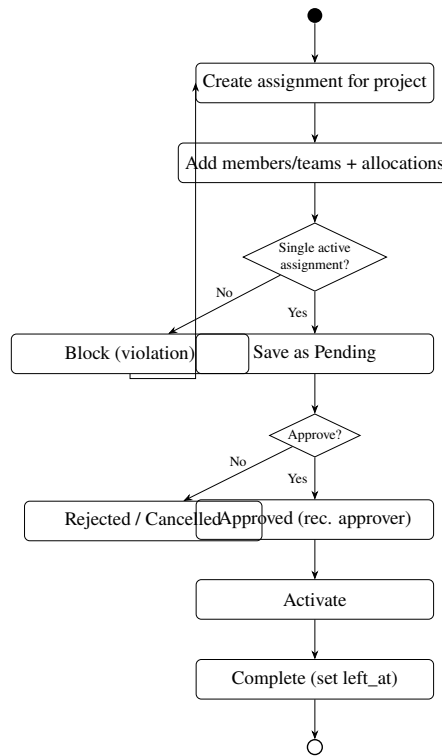


Fig. 3.6: Activity diagram — Assignment status workflow

3.3.6 Use Case 6: View Dashboard Analytics

Tab. 3.9: Use Case: View Dashboard Analytics

Field	Description
Title	View Dashboard Analytics
Actors	All authenticated users
Description	Dashboard displays key metrics: total employees by department, workload vs availability chart, projects by status, projects needing staffing, over-allocated employees, expiring certifications, and recent activity log.
Pre-conditions	User authenticated.
Post-conditions	Dashboard displays current state of organization and assignments.
Main Flow	1. User navigates to dashboard (Filament panel for admin users, public dashboard for regular users) 2. System loads StatsOverview widget (total counts) 3. System loads EmployeesByDepartmentChart (bar chart) 4. System loads AllocationVsAvailabilityChart (grouped bar: workload vs availability per department) 5. System loads ProjectsByStatusChart (doughnut chart) 6. System loads alert tables: ProjectsNeedingStaffing, OverAllocatedEmployees, ExpiringCertifications 7. System loads RecentActivityTable (audit log entries)
Alternative Flow	Empty data states show placeholder messages. Charts are rendered client-side via Filament ChartWidget.

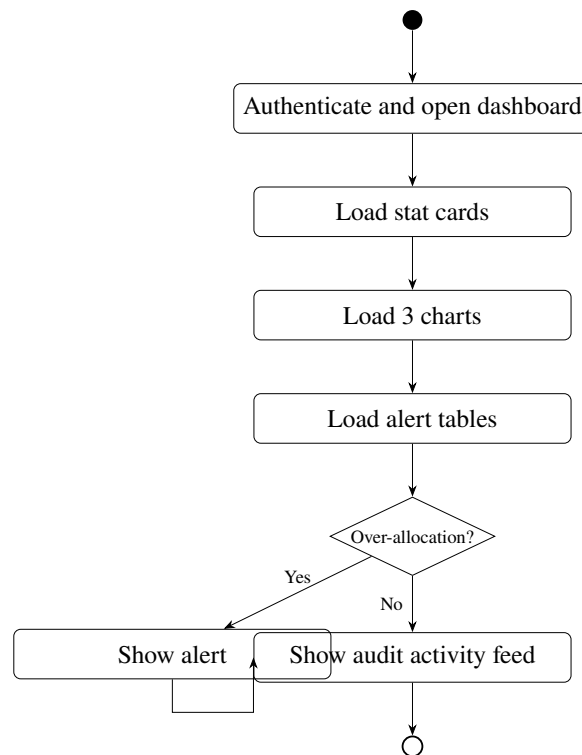


Fig. 3.7: Activity diagram — View dashboard analytics

3.3.7 Use Case 7: Audit Trail

Tab. 3.10: Use Case: Audit Trail

Field	Description
Title	Audit Trail
Actors	Super Admin, Admin
Description	Every assignment action (created, approved, activated, completed, rejected, cancelled) is recorded in audit_logs with actor ID, action, auditable type and ID, old/new values, IP address, and user agent.
Pre-conditions	AuditService is injected into AssignmentService.
Post-conditions	Audit log entry is created for every state transition.
Main Flow	1. Assignment state transition occurs (e.g., approve) 2. AssignmentService calls AuditService.assignmentTransition() 3. AuditService creates AuditLog record with action, old_values, new_values, IP, user_agent 4. Record is persisted to audit_logs table
Alternative Flow	Manual log entries can be created via AuditService.log() for any model and action.

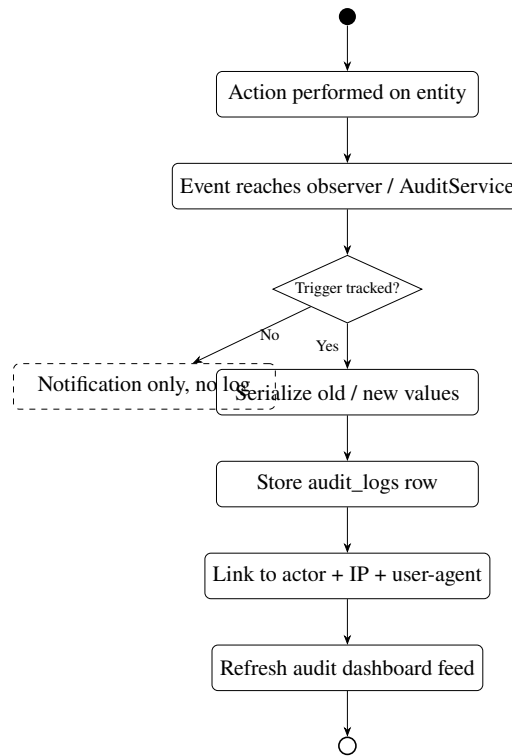


Fig. 3.8: Activity diagram — Audit trail

3.3.8 Use Case 8: Run Recommendation for Staffing

Tab. 3.11: Use Case: Run Recommendation for Staffing

Field	Description
Title	Run Recommendation for Staffing
Actors	Super Admin, Admin, Resource Manager
Description	The system ranks the candidate employees of a project against its typed requirements and produces an ordered recommendation list with scores. The recommendation can operate in three modes: employee, team, or department.
Pre-conditions	User authenticated with recommendation permissions. The project exists in the Staffing state and carries typed requirements.
Post-conditions	A RecommendationRun is persisted together with the ranked Recommendations; the ordered list is displayed to the resource manager.
Main Flow	1. RM opens a project in the Staffing state and selects “Run recommendation” 2. RM chooses the mode (employee, team, or department) 3. The system evaluates every candidate against the weighted typed requirements (skill, certification, language, experience, position, department, location, availability) 4. The candidates are scored and sorted in descending order 5. The system persists the RecommendationRun and the recommendations, then displays the ranked list
Alternative Flow	If no candidate satisfies the minimum coverage, an empty result is displayed with an explanatory message.

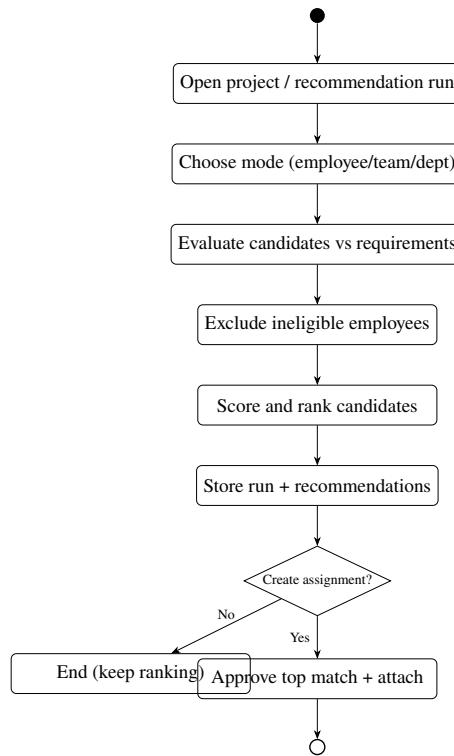


Fig. 3.9: Activity diagram — Run recommendation for staffing

3.3.9 Use Case 9: Submit and Approve Profile Change Requests

Tab. 3.12: Use Case: Submit and Approve Profile Change Requests

Field	Description
Title	Submit and Approve Profile Change Requests
Actors	Employee (submits), HR Manager / Admin (approves)
Description	Through the self-service portal, an employee requests updates to their own profile (contact, skills, availability, and so on). The changes are applied only after the approval of an HR manager, keeping the referential under control.
Pre-conditions	The employee is authenticated on the portal and the requested fields are <u>editable</u> .
Post-conditions	The approved changes are applied to the employee profile and the <u>whole flow is recorded in the audit trail</u> .
Main Flow	1. The employee opens the profile page of the portal 2. The employee submits a change request with the new values and a reason 3. The system stores a ProfileChangeRequest in the Pending state 4. An HR manager reviews the request 5. If approved, the system applies the changes and records the audit; otherwise it rejects the request with a reason
Alternative Flow	The employee may cancel an own request that has not yet been reviewed.

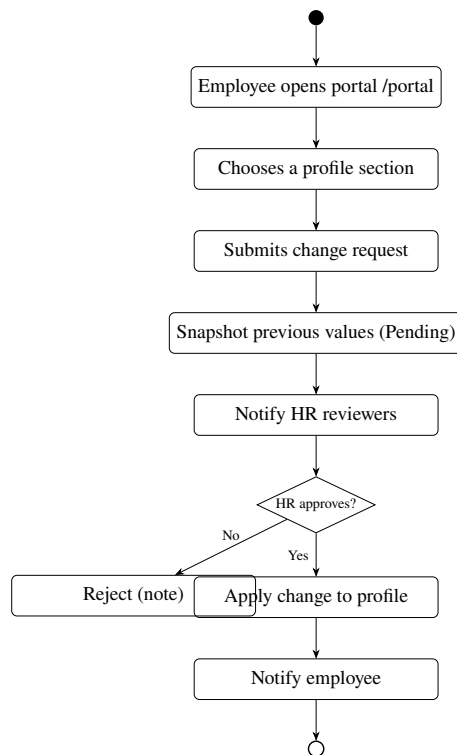


Fig. 3.10: Activity diagram — Submit and approve profile change requests

3.4 Dynamic Study — Sequence Diagrams

The sequence diagrams below describe the exchange of messages between the actors and the application objects for the main scenarios of the platform. Figure 3.11 details the authentication, Figure 3.12 the management of the organization, Figure 3.13 the creation of an employee, Figure 3.14 the creation of a project, Figure 3.15 the assignment lifecycle, Figure 3.16 the display of the dashboard, Figure 3.17 the audit capture, Figure 3.18 the approval of a profile change request, and Figure 3.19 a recommendation run.

3.5 Class Diagram

The class diagram models the static structure of the Djezzy platform, identifying the main domain entities, their attributes, methods, and relationships.

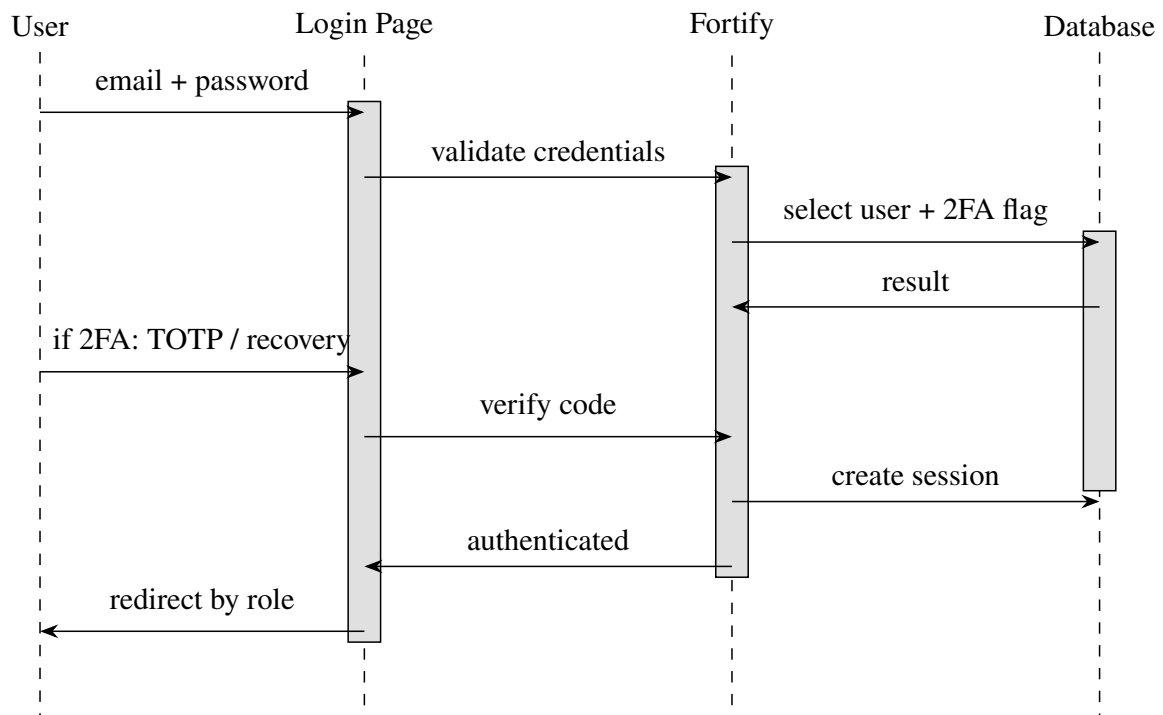


Fig. 3.11: Sequence diagram — Authentication with 2FA/passkeys

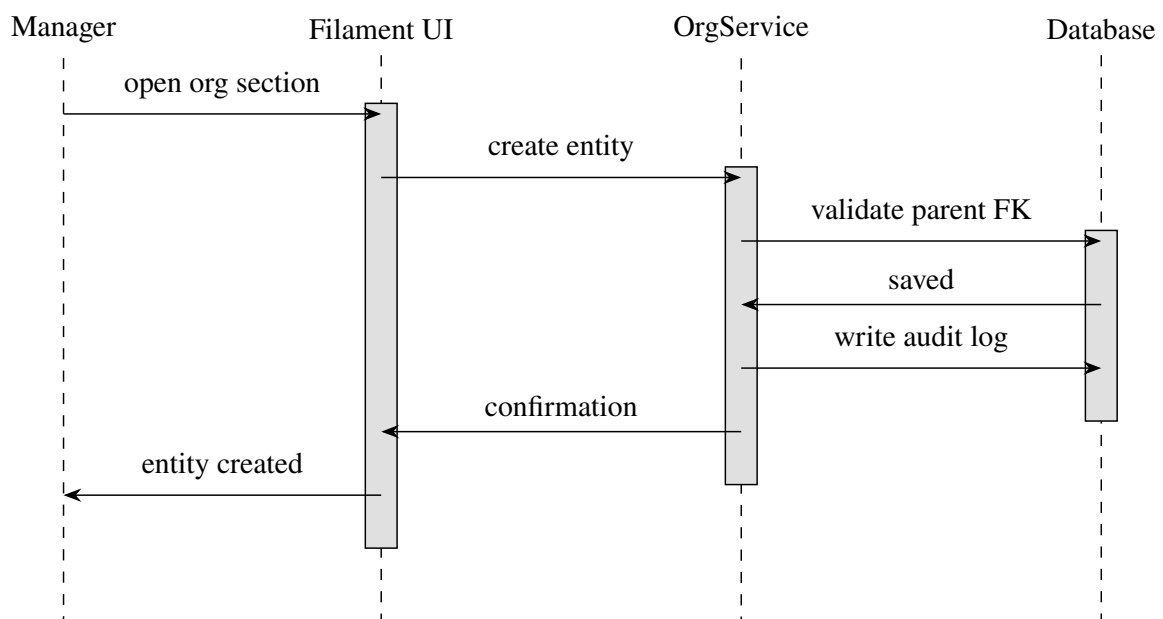


Fig. 3.12: Sequence diagram — Manage organization structure

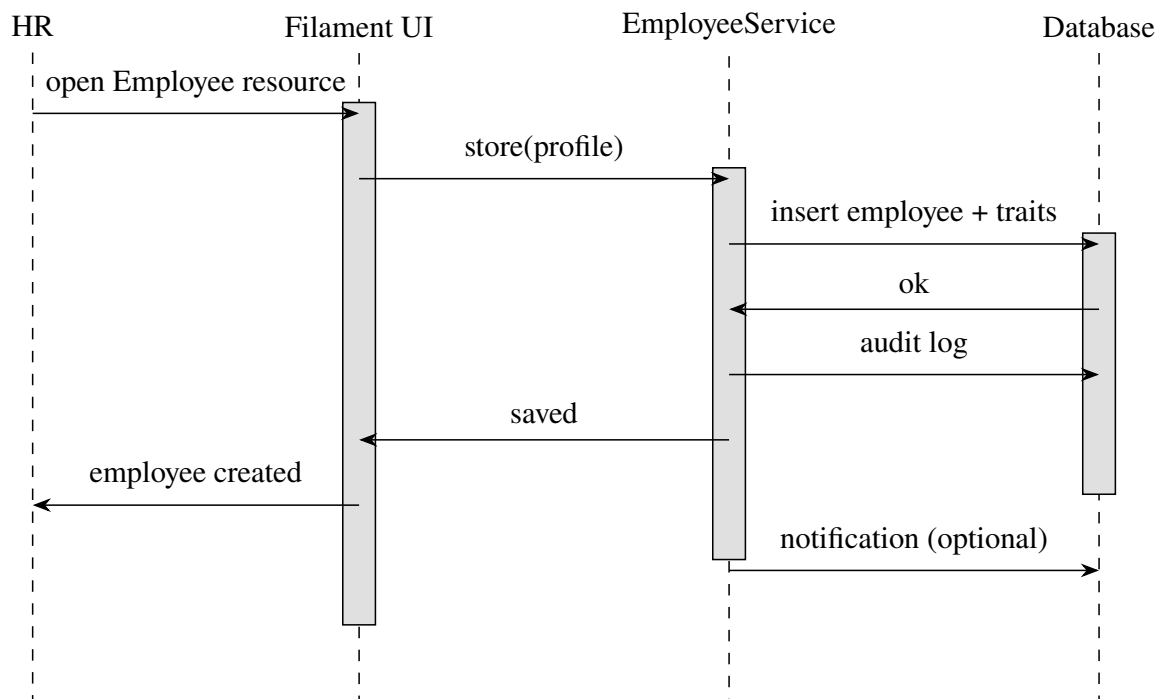


Fig. 3.13: Sequence diagram — Create employee

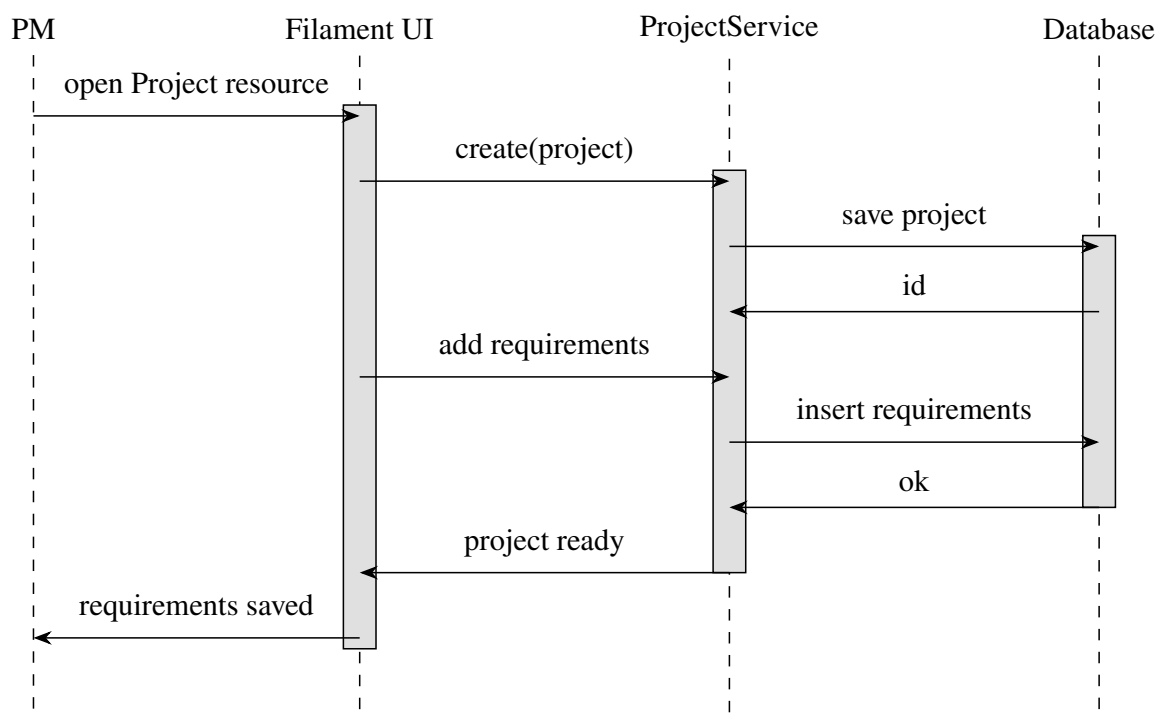


Fig. 3.14: Sequence diagram — Create project with requirements

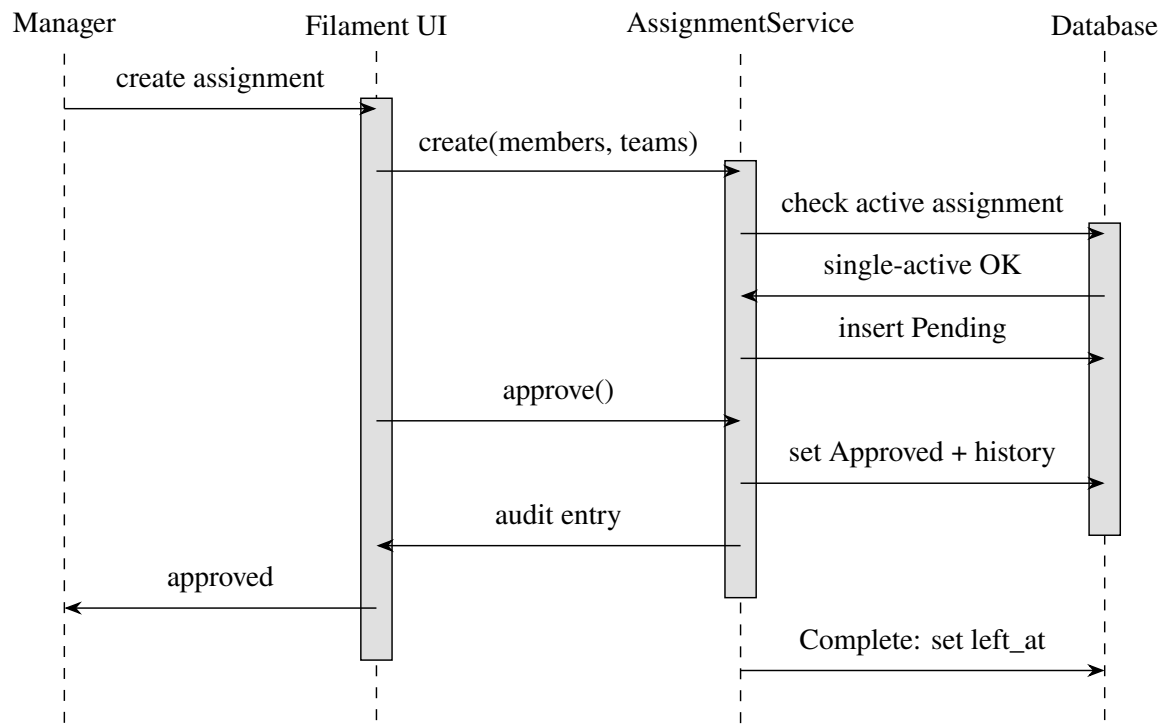


Fig. 3.15: Sequence diagram — Assignment status workflow

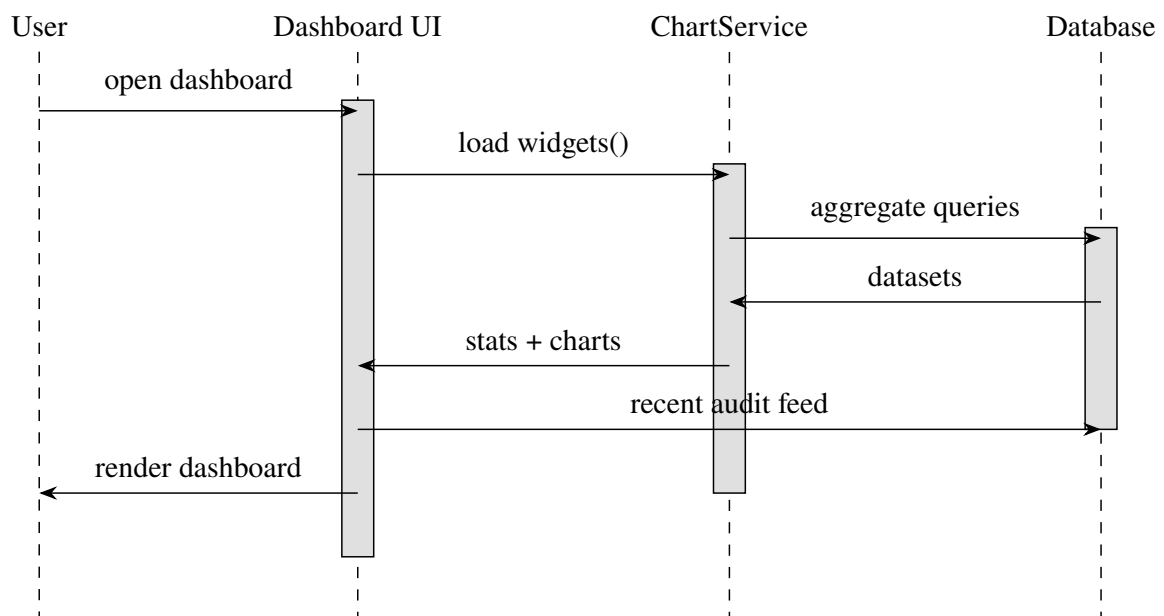


Fig. 3.16: Sequence diagram — Dashboard analytics

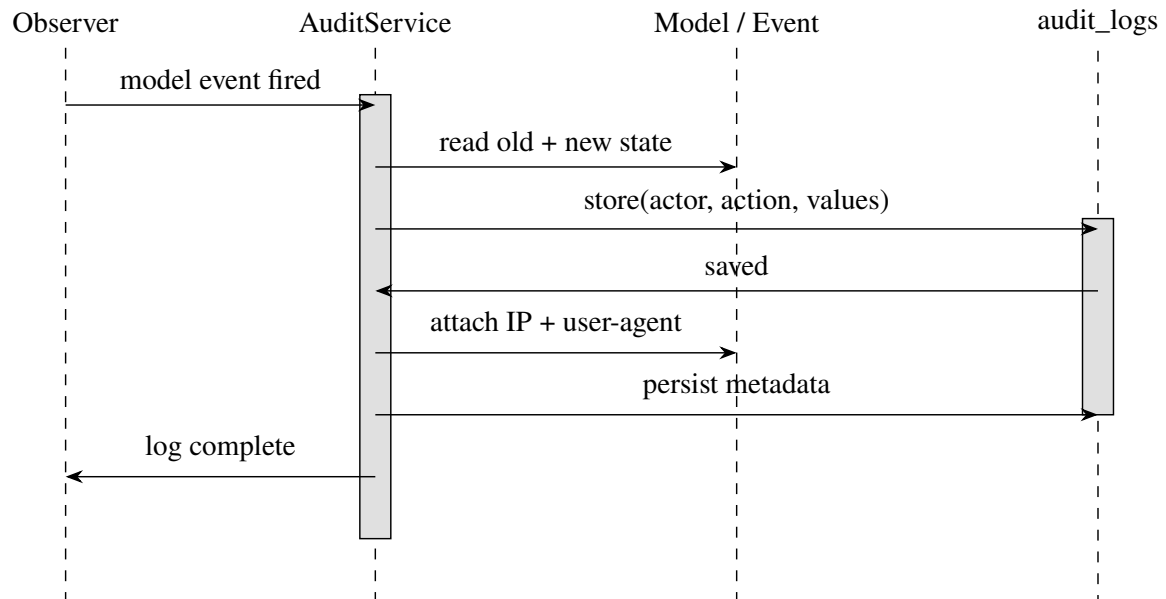


Fig. 3.17: Sequence diagram — Audit trail

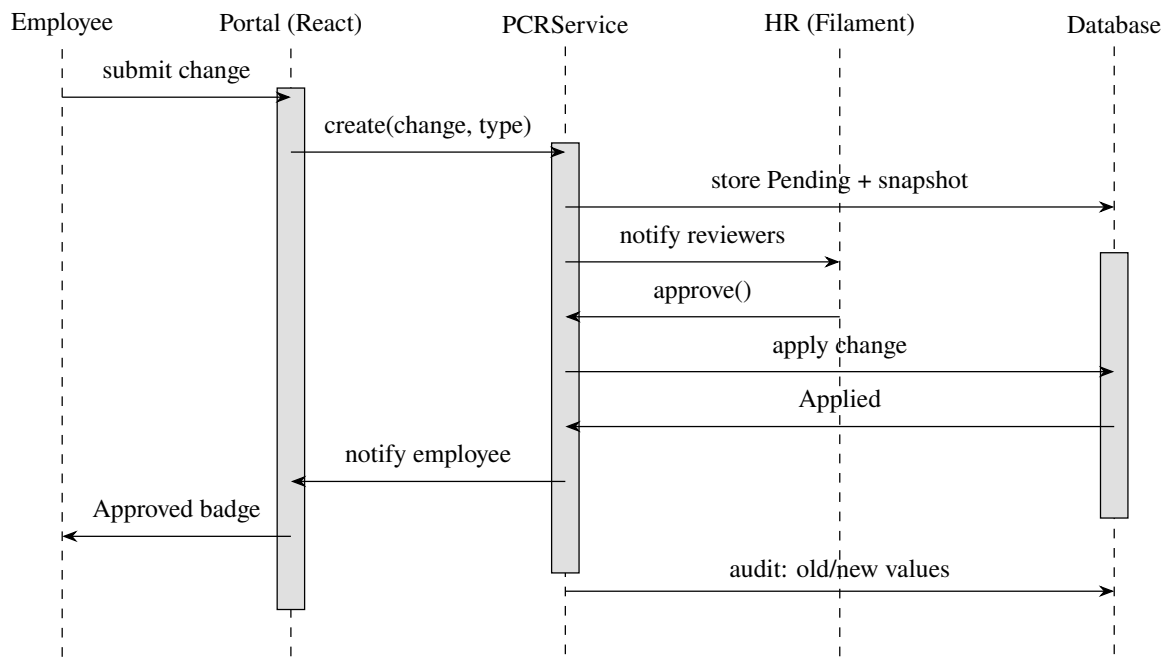


Fig. 3.18: Sequence diagram — Profile change request approval

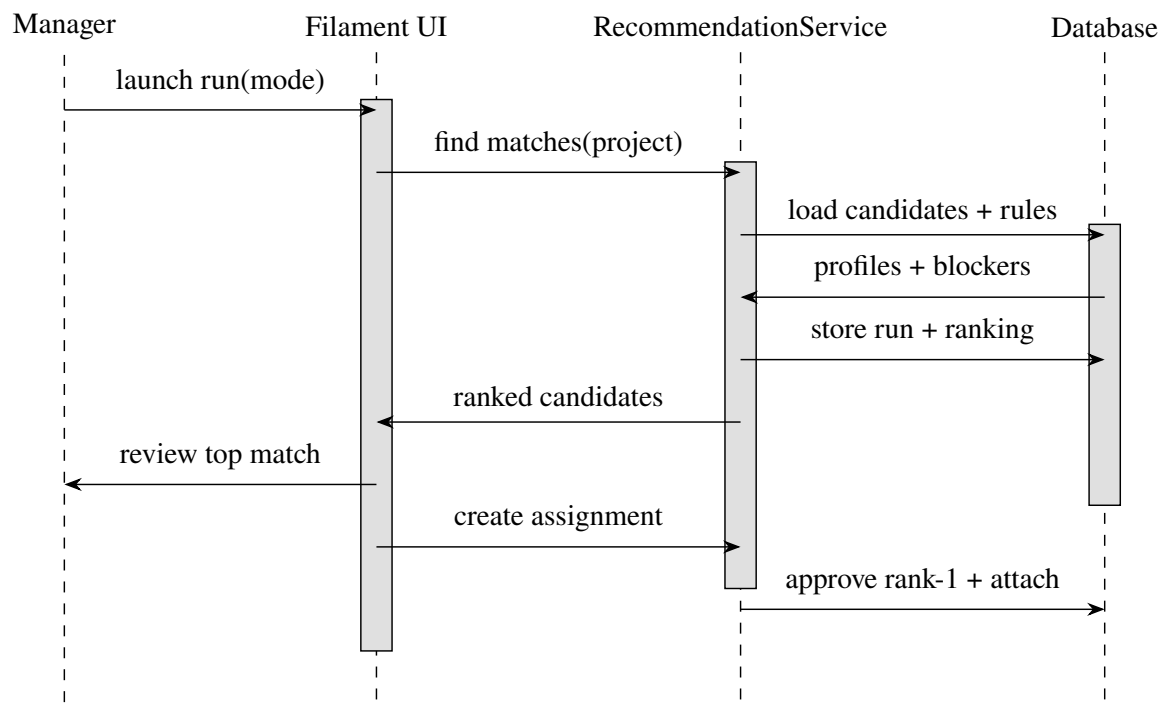


Fig. 3.19: Sequence diagram — Recommendation run

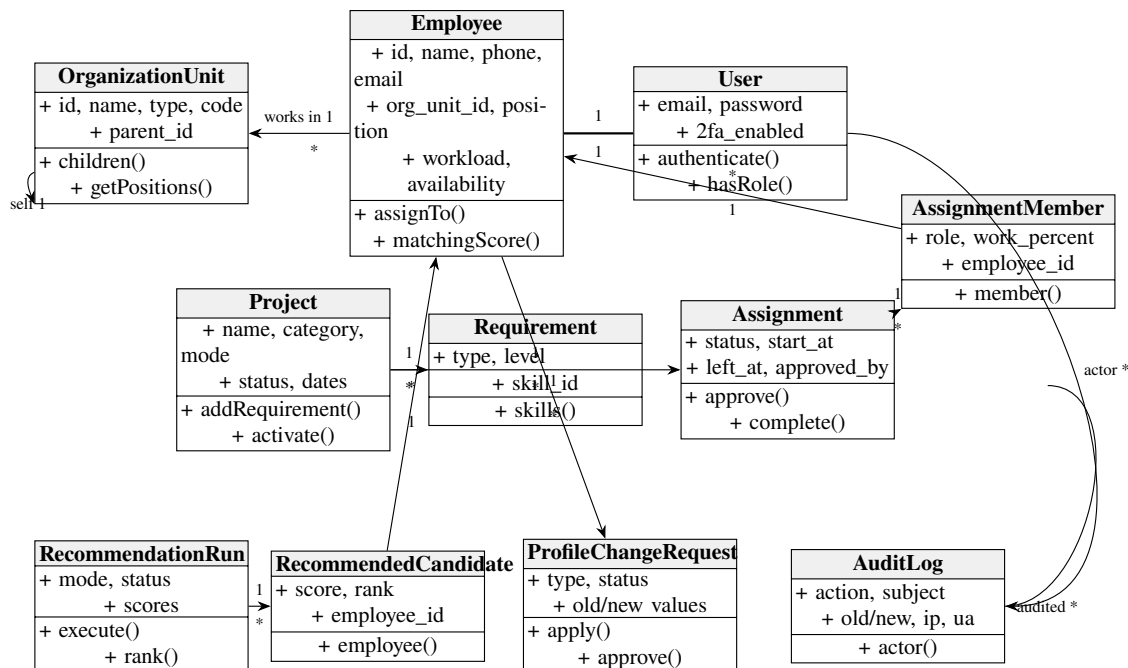


Fig. 3.20: Class Diagram — Domain Model

3.5.1 Class Descriptions

Tab. 3.13: Class: User

Class	Attributes	Methods
Attributes	id, name, email, password, email_verified_at, two_factor_secret, two_factor_recovery_codes, remember_token, created_at, updated_at	
Methods	assignRole(), hasRole(), can(), delete()	

Tab. 3.14: Class: Employee

Class	Attributes	Methods
Attributes	id, user_id, matricule, first_name, last_name, email, phone, hire_date, employment_status, employment_type, department_id, position_id, location_id, team_id, manager_id, created_at, updated_at, deleted_at	
Methods	skills(), certifications(), languages(), availabilities(), workloads(), assignments(), fullName()	

Tab. 3.15: Class: Department

Class	Attributes	Methods
Attributes	id, business_unit_id, name, code, description, head_id, created_at, updated_at, deleted_at	
Methods	businessUnit(), employees(), teams()	

Tab. 3.16: Class: Project

Class	Attributes	Methods
Attributes	id, name, code, description, project_category_id, assignment_mode, priority, status, start_date, target_end_date, actual_end_date, estimated_effort_hours, confidentiality_level, requested_by, project_manager_id, created_at, updated_at, deleted_at	
Methods	assignments(), requirements(), category(), evaluations()	

Tab. 3.17: Class: Assignment

Class	Attributes	Methods
Attributes	id, project_id, assigned_by, approved_by, assignment_type, status, assigned_at, approved_at, start_date, end_date, notes, created_at, updated_at	
Methods	project(), members(), teams(), statusHistory()	

Tab. 3.18: Class: Skill

Class	Attributes	Methods
Attributes	id, skill_category_id, name, code, description, created_at, updated_at, deleted_at	
Methods	category(), employees(), projects()	

Tab. 3.19: Class: Certification

Class	Attributes	Methods
Attributes	id, name, code, issuer, validity_months, created_at, updated_at, deleted_at	
Methods	employees()	

Tab. 3.20: Class: Language

Class	Attributes	Methods
Attributes	id, name, code, created_at, updated_at, deleted_at	
Methods	employees()	

Tab. 3.21: Class: AuditLog

Class	Attributes	Methods
Attributes	id, user_id, action, auditable_type, auditable_id, old_values, new_values, ip_address, user_agent, created_at	
Methods	user(), auditable()	

Tab. 3.22: Class: AssignmentStatusHistory

Class	Attributes	Methods
Attributes	id, assignment_id, old_status, new_status, changed_by, reason, created_at	
Methods	assignment(), changedByUser()	

3.5.2 Passage to the Relational Model

The transformation of the UML class diagram into a relational schema follows the classical rules of passage to the relational model. For a BTS-type implementation these rules are applied as follows:

- **Class** → **table**: every persistent class becomes a table; each attribute becomes a column and the primary key (PK) of the class becomes the primary key of the table.
- **Association 1..1 / 1..***: the "one" side is represented by adding a foreign key (FK) in the table of the "many" side. Example: `employees.department_id` references `departments`.
- **Association N..M**: an N-M association cannot be represented directly; it is materialized by a **pivot (junction) table** containing the primary keys of the two associated tables, together with the attributes of the association. Example: `employee_skills`.
- **Inheritance**: the User class is materialized by a `users` table; its specialization `Employee` adds a foreign key `user_id` referencing `users` (simple inheritance by reference).
- **Attributes of associations**: attributes carried by an association (for example the proficiency level of a skill, the allocation percentage of an assignment member) are placed in the pivot table.

The following table gives the complete mapping of the fifteen domain classes of the class diagram onto the relational schema:

Tab. 3.23: Mapping — UML classes to relational tables

UML Class	SQL Table	PK	FK	Remarks
User	users	id	—	Authentication and identity
Employee	employees	id	user_id, department_id, position_id, location_id, team_id, manager_id	Specialization of User
Department	departments	id	business_unit_id, head_id	—
BusinessUnit	business_units	id	—	One-to-many with departments
Team	teams	id	department_id, team_lead_id	—
Position	positions	id	—	—
Location	locations	id	—	—
Skill	skills	id	skill_category_id	—
Certification	certifications	id	—	—
Language	languages	id	—	—
Project	projects	id	project_category_id, project_manager_id	—
Assignment	assignments	id	project_id, assigned_by, approved_by	Status machine table
AssignmentStatusHistory	assignment_status_history	id	assignment_id, changed_by	History of transitions
AuditLog	audit_logs	id	user_id	Polymorphic target

The N-M associations are materialized by the following pivot tables:

Tab. 3.24: Pivot tables created from the N-M associations

Pivot Table	Association	Foreign Keys	Attributes Carried
employee_skills	Employee N–M Skill	employee_id, skill_id	proficiency_level (1–5)
employee_certifications	Employee N–M Certification	employee_id, certification_id	obtained_date, verification_status
employee_languages	Employee N–M Language	employee_id, language_id	proficiency_level (1–5)
requirement_skills	Project N–M Skill	project_id, skill_id	required_level, weight
requirement_certifications	Project N–M Certification	project_id, certification_id	weight
requirement_languages	Project N–M Language	project_id, language_id	required_level, weight
assignment_members	Assignment N–M Employee	assignment_id, employee_id	allocation_percentage, responsibility, start_date, end_date
assignment_teams	Assignment N–M Team	assignment_id, team_id	—

3.6 Database Schema

The database schema consists of 44 tables in total, grouped by concern (authentication and security, RBAC, organization, employees, catalog, projects, assignments, portal, audit, recommendations). This section presents the eight core tables that form the foundation of the platform’s data model; the complete inventory is given at the end of the section.

3.6.1 Table: users

Tab. 3.25: Table: users

Column	Type	Nullable	Description
id	BIGINT	No	Primary key
name	VARCHAR(255)	No	Full name
email	VARCHAR(255)	No	Unique email address
password	VARCHAR(255)	No	Hashed password
email_verified_at	TIMESTAMP	Yes	Email verification timestamp
two_factor_secret	TEXT	Yes	Encrypted 2FA TOTP secret
two_factor_recovery_codes	TEXT	Yes	Encrypted recovery codes
remember_token	VARCHAR(100)	Yes	Remember me token

3.6.2 Table: employees

Tab. 3.26: Table: employees

Column	Type	Nullable	Description
id	BIGINT	No	Primary key
user_id	BIGINT	Yes	FK to users (nullable)
matricule	VARCHAR(50)	No	Unique employee ID
first_name	VARCHAR(255)	No	First name
last_name	VARCHAR(255)	No	Last name
email	VARCHAR(255)	Yes	Work email
phone	VARCHAR(50)	Yes	Phone number
hire_date	DATE	No	Hire date
employment_status	ENUM	No	active, on_leave, suspended, re-signed, retired
employment_type	ENUM	No	full_time, part_time, contractor, intern
department_id	BIGINT	No	FK to departments
position_id	BIGINT	No	FK to positions
location_id	BIGINT	Yes	FK to locations
team_id	BIGINT	Yes	FK to teams
manager_id	BIGINT	Yes	FK to employees (self-referencing)
deleted_at	TIMESTAMP	Yes	Soft delete timestamp

3.6.3 Table: assignments

Tab. 3.27: Table: assignments

Column	Type	Nullable	Description
id	BIGINT	No	Primary key
project_id	BIGINT	No	FK to projects
assigned_by	BIGINT	No	FK to users
approved_by	BIGINT	Yes	FK to users (approver)
assignment_type	ENUM	No	employee, team, department, mixed
status	ENUM	No	pending, approved, active, completed, rejected, cancelled
assigned_at	TIMESTAMP	No	Assignment creation timestamp
approved_at	TIMESTAMP	Yes	Approval timestamp
start_date	DATE	Yes	Assignment start date
end_date	DATE	Yes	Assignment end date
notes	TEXT	Yes	Free-form notes
created_at	TIMESTAMP	No	Record creation timestamp
updated_at	TIMESTAMP	No	Record update timestamp

3.6.4 Table: assignment_members

Tab. 3.28: Table: assignment_members

Column	Type	Nullable	Description
id	BIGINT	No	Primary key
assignment_id	BIGINT	No	FK to assignments (cascade delete)
employee_id	BIGINT	No	FK to employees (restrict delete)
responsibility	VARCHAR(255)	Yes	Role description
allocation_percentage	DECIMAL(5,2)	Yes	Allocation % (CHECK 0–100)
allocated_hours	DECIMAL(10,2)	Yes	Allocated hours
joined_at	DATE	Yes	Join date
left_at	DATE	Yes	Leave date (set on completion)

3.6.5 Table: audit_logs

Tab. 3.29: Table: audit_logs

Column	Type	Nullable	Description
id	BIGINT	No	Primary key
user_id	BIGINT	No	FK to users (actor)
action	VARCHAR(255)	No	Action identifier (e.g. assignment.approved)
auditable_type	VARCHAR(255)	No	Morph type
auditable_id	BIGINT	No	Morph ID
old_values	JSON	Yes	Previous state
new_values	JSON	Yes	New state
ip_address	VARCHAR(45)	Yes	Client IP address
user_agent	TEXT	Yes	Client user agent string

3.6.6 Table: projects

Tab. 3.30: Table: projects

Column	Type	Nullable	Description
id	BIGINT	No	Primary key
project_category_id	BIGINT	No	FK to project_categories
name	VARCHAR(255)	No	Project name
code	VARCHAR(50)	No	Unique project code (codification)
description	TEXT	Yes	Project description
assignment_mode	ENUM	No	employee, team, department, mixed
priority	ENUM	No	low, medium, high, critical
status	ENUM	No	draft, submitted, under_review, staffing, assigned, in_progress, completed
start_date	DATE	Yes	Project start date
end_date	DATE	Yes	Project end date
confidential	BOOLEAN	No	Confidentiality level
created_at	TIMESTAMP	No	Record creation timestamp
updated_at	TIMESTAMP	No	Record update timestamp

3.6.7 Table: profile_change_requests

Tab. 3.31: Table: profile_change_requests

Column	Type	Nullable	Description
id	BIGINT	No	Primary key
employee_id	BIGINT	No	FK to employees
requested_by	BIGINT	No	FK to users (the employee)
reviewed_by	BIGINT	Yes	FK to users (the reviewer)
changes_data	JSON	No	Requested field/value pairs
reason	TEXT	Yes	Reason given by the employee
status	ENUM	No	pending, approved, rejected, cancelled
review_comment	TEXT	Yes	Reviewer comment
reviewed_at	TIMESTAMP	Yes	Review timestamp
created_at	TIMESTAMP	No	Record creation timestamp
updated_at	TIMESTAMP	No	Record update timestamp

3.6.8 Table: recommendation_runs

Tab. 3.32: Table: recommendation_runs

Column	Type	Nullable	Description
id	BIGINT	No	Primary key
project_id	BIGINT	No	FK to projects
run_by	BIGINT	No	FK to users
mode	ENUM	No	employee, team, department
status	ENUM	No	pending, completed, failed
summary	JSON	Yes	Execution summary
created_at	TIMESTAMP	No	Record creation timestamp
updated_at	TIMESTAMP	No	Record update timestamp

3.6.9 Complete inventory of the database

The forty-four tables of the platform are inventoried below, grouped by functional concern. Each table is accompanied by a short description of its purpose.

Tab. 3.33: Inventory — Authentication, security and RBAC group

Table	Purpose
users	Application users (admin staff and employees)
password_reset_tokens	Password reset tokens (Fortify)
sessions	Authenticated sessions
jobs	Queued jobs
job_batches	Job batches
failed_jobs	Failed job persistence
cache	Application cache store
cache_locks	Cache locks
roles	Role labels and guards (6 roles)
permissions	Permissions labels and guards (52 permissions)
model_has_roles	Polymorphic user–role association
model_has_permissions	Polymorphic user–permission overrides
role_has_permissions	Permissions granted to each role

Tab. 3.34: Inventory — Organization and employees group

Table	Purpose
business_units	Top-level organizational divisions
departments	Departments belonging to business units
teams	Teams belonging to departments
positions	Job positions (titles)
locations	Work locations and sites
team_members	Employees composing each team
employees	Employee profiles, organization references and employment data
skills	Catalog of skill labels
employee_skills	Employee skills with proficiency (1–5)
certifications	Catalog of certifications
employee_certifications	Employee certifications with verification status
languages	Catalog of languages
employee_languages	Employee languages with proficiency (1–5)
availabilities	Employee availability records (status, dates)
workloads	Employee workload percentages (CHECK 0–100)

Tab. 3.35: Inventory — Catalog, projects and assignments group

Table	Purpose
project_categories	Project classifications
projects	Project records with status and assignment mode
requirements	Typed requirements of a project (skill, certification, language, experience, position, department, location, availability)
requirement_skills	Project–skill requirement pivots (level, weight)
requirement_certifications	Project–certification requirement pivots (level, weight)
requirement_languages	Project–language requirement pivots (level, weight)
assignments	Assignment lifecycle with status machine
assignment_members	Assignment–employee links with allocation, responsibility and dates
assignment_teams	Assignment–team links
assignment_status_history	Status transitions of an assignment (actor, reason)

Tab. 3.36: Inventory — Portal, audit and recommendations group

Table	Purpose
profile_change_requests	Employee self-service change requests awaiting approval
profile_change_request_fields	Requested field values of each change request
notifications	In-application notifications
audit_logs	Complete audit trail (old/new values, IP, user agent)
recommendation_runs	Recommendation engine executions
recommendations	Ranked and scored candidates of a run

3.7 Deployment Diagram

The Djezzy platform follows a monolithic deployment architecture on a standard LEMP (Linux, Nginx, MySQL/PostgreSQL, PHP) stack:

- **Web Server:** Apache/Nginx with PHP-FPM for request processing
- **Application Server:** Laravel 13 running on PHP 8.3 with all application logic
- **Filament Admin Panel:** Served at the /admin path with role-based access
- **React/Inertia Frontend:** Server-side rendered React components via Inertia.js
- **Database:** PostgreSQL recommended for production; SQLite for local development and testing
- **Asset Build:** Vite 8 for JavaScript/CSS compilation with hot module replacement during development
- **Authentication:** Laravel Fortify with optional 2FA and passkey support

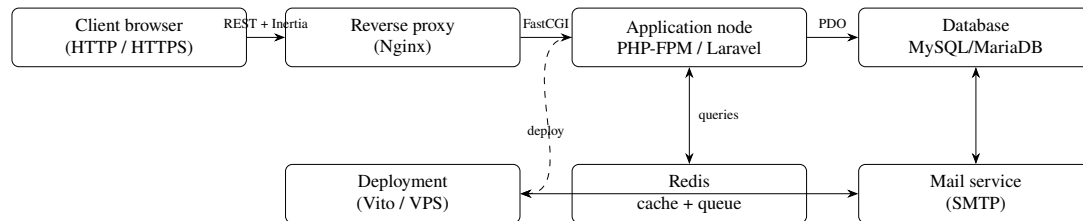


Fig. 3.21: Deployment Diagram — Server Infrastructure

Conclusion

This chapter has presented the complete analysis and design of the Djezzy Resource Assignment Platform. Nine use cases have been fully described with their main and alternative flows, each accompanied by an activity diagram, and the dynamic study details nine sequence diagrams covering the principal scenarios. The codification rules and the eleven rules of management (RG01–RG11) frame the business constraints. The class diagram identifies ten domain classes with their attributes and methods. The database schema defines eight core tables with complete column specifications and a full inventory of the forty-four tables grouped by concern. The deployment architecture follows a proven monolithic pattern with Laravel, Filament, and PostgreSQL. These design artifacts provide a clear blueprint for the implementation phase detailed in the next chapter.

CHAPTER IV

Implementation and Validation

Introduction

This chapter details the implementation of the Djezzy Resource Assignment Platform, presenting the development tools and technologies used, the application structure, the key user interfaces, and the testing and validation approach that ensures the platform meets its requirements.

4.1 Languages and Tools

The development of the platform required a comprehensive technology stack spanning frontend, backend, database, testing, and design tools. The main technologies are presented below, each with its role in the platform and its official logo.

Tab. 4.1: Development Languages and Tools — Summary

Tool	Description
PHP 8.3	Server-side scripting language powering Laravel
Laravel 13.17	PHP MVC framework with Eloquent ORM, migrations, middleware
Filament 5.1	Admin panel builder for Laravel with CRUD resources, widgets, charts
React 19	Component-based JavaScript library for frontend UI
TypeScript	Typed superset of JavaScript for type safety
Inertia.js 3.0	Server-driven SPA bridge between Laravel routing and React rendering
Tailwind CSS 4	Utility-first CSS framework for rapid UI development
Vite 8.0	Modern build tool with HMR and optimized production builds
PostgreSQL	Recommended production relational database with partial unique indexes
SQLite	Lightweight database used for local development and testing
Spatie Laravel Permission	Role-based access control package with granular permissions
Laravel Fortify	Authentication scaffolding with 2FA and passkey support
PHPUnit / Pest	PHP testing frameworks for unit and feature tests
Radix UI	Headless UI component primitives for React (dialogs, dropdowns, tooltips)
Lucide React	Icon library for React applications
Sonner	Toast notification library for React
Draw.io	Diagram creation tool for UML and architecture diagrams

4.1.1 PHP 8.3

PHP is a server-side scripting language specifically suited for web development. Version 8.3 adds typed class constants, garbage collection improvements, and performance optimizations. PHP

was chosen because it is the native language of the Laravel framework and remains one of the most deployed languages in the web hosting ecosystem, guaranteeing long-term maintainability of the platform.



Fig. 4.1: PHP — Server-Side Scripting Language

4.1.2 Laravel 13

Laravel is a full-stack PHP MVC framework providing an expressive syntax, the Eloquent ORM, migrations, middleware, queues, and an extensive package ecosystem. It was chosen for its rapid development workflow, its strong default security (password hashing, CSRF protection, SQL injection protection through the query builder), and its support for long-term releases.



Fig. 4.2: Laravel — The PHP Framework for Web Artisans

4.1.3 Filament 5

Filament is an open-source admin panel builder for Laravel. It generates CRUD resources, forms, tables, widgets, and dashboards from PHP configuration files without writing frontend code. It was selected because it allowed the rapid construction of the six administration modules (User, Employee, Department, Project, Assignment, Audit) while respecting the RBAC permission matrix through integrated authorization.



Fig. 4.3: Filament — Laravel Admin Panel Builder

4.1.4 React 19

React is a component-based JavaScript library for building user interfaces. Version 19 introduces the compiler, improved server components, and better memoization. React was chosen for the employee self-service portal because of its component reuse, its large ecosystem, and its maintainability by the frontend team of Djezzy.



Fig. 4.4: React — User Interface Library

4.1.5 TypeScript

TypeScript is a typed superset of JavaScript that compiles to plain JavaScript. It was used on the React side to declare interfaces for the Inertia page props (for example the shape of the candidate scores returned by the recommendation engine), which eliminates an entire category of runtime errors during development.



Fig. 4.5: TypeScript — Typed Superset of JavaScript

4.1.6 Inertia.js 3

Inertia.js is a bridge that connects the server-side routing of Laravel to the client-side rendering of React, without building a separate REST API. Each Inertia response describes a "page" (component name plus props) that React reconstructs as an SPA. This architecture was chosen to keep the authorization logic centralized in Laravel while offering a fluid, navigation-without-reload experience in the portal.



Fig. 4.6: Inertia.js — Server-Driven SPA Bridge for Laravel

4.1.7 Tailwind CSS 4

Tailwind CSS is a utility-first CSS framework. Version 4 uses a new CSS-based configuration engine and automatic content detection. Tailwind was chosen to style the Filament panel and the portal consistently, with design tokens (colors, spacing, typography) defined in one place, speeding up UI iteration.

Tailwind CSS

Fig. 4.7: Tailwind CSS — Utility-First CSS Framework

4.1.8 Vite 8

Vite is a modern build tool offering fast cold-start development servers and Hot Module Replacement (HMR). It powers the React/TypeScript pipeline of the platform, bundles the production assets, and integrates with Laravel through the official Vite plugin, so that the compiled JavaScript and CSS are hashed and versioned automatically.

Vite

Fig. 4.8: Vite — Frontend Build Tool

4.1.9 PostgreSQL

PostgreSQL is an advanced open-source relational database management system. It was chosen for the production environment for its referential integrity, its partial unique indexes (used to enforce the single active assignment of an employee), its full-text search capabilities, and its transactional guarantees (ACID). SQLite is used at development and test time because it requires no server setup and runs in-memory for the test suites.

PostgreSQL

Fig. 4.9: PostgreSQL — Relational Database Management System

4.1.10 PHPUnit and Pest

PHPUnit is the reference unit-testing framework for PHP, and Pest is a thin expressive layer on top of it. The platform contains 68 tests organized in 16 files, covering authentication, authorization, domain rules, and the assignment workflow. They can be executed with `phpartisantest` in a few seconds against a SQLite in-memory database.

PHPUnit / Pest

Fig. 4.10: PHPUnit and Pest — Testing Frameworks

4.1.11 Supporting Libraries and Tools

Several supporting packages complete the stack. **Spatie Laravel Permission** implements the role-based access control model (6 roles, 52 permissions) used by every administration module. **Laravel Fortify** provides the authentication scaffolding, including two-factor authentication (2FA) and passkeys. On the React side, **Radix UI** supplies headless accessible components (dialogs, dropdowns, tooltips), **Lucide React** provides the icons, and **Sonner** handles toast notifications. Finally, **Draw.io** was used to create the UML diagrams of Chapter III.

4.2 Application Structure

The application follows Laravel’s conventional directory structure, extended for the Filament admin panel and Inertia/React frontend:

- `app/Models/`: Eloquent models (User, Employee, Department, Project, Assignment, Skill, Certification, Language, AuditLog, AssignmentStatusHistory, and 27 additional models)
- `app/Filament/`: Filament admin panel resources, pages, widgets, and panels organized by domain (Organization, HumanResources, Projects, Assignments)
- `app/Services/`: Business logic services (AssignmentService, AuditService) encapsulating domain rules and state transitions
- `app/Http/`: Controllers for Inertia/React routes (settings, profile)
- `resources/js/`: React/TypeScript components for Inertia pages (settings, profile, auth)
- `database/migrations/`: 37 migration files defining the complete database schema
- `database/seeders/`: Database seeders providing initial data (roles, permissions, sample organizations)
- `tests/Feature/`: Feature tests organized by domain (Auth, Settings, Domain, Admin, Dashboard)

4.2.1 Architecture and Request Flow

The application implements the layered architecture already introduced in Figure 4.11: a presentation layer (Filament resources and Inertia pages), an application layer made of services, policies and form requests, a domain layer constituted by the Eloquent models, and an infrastructure layer composed of the database, the cache and the audit service.

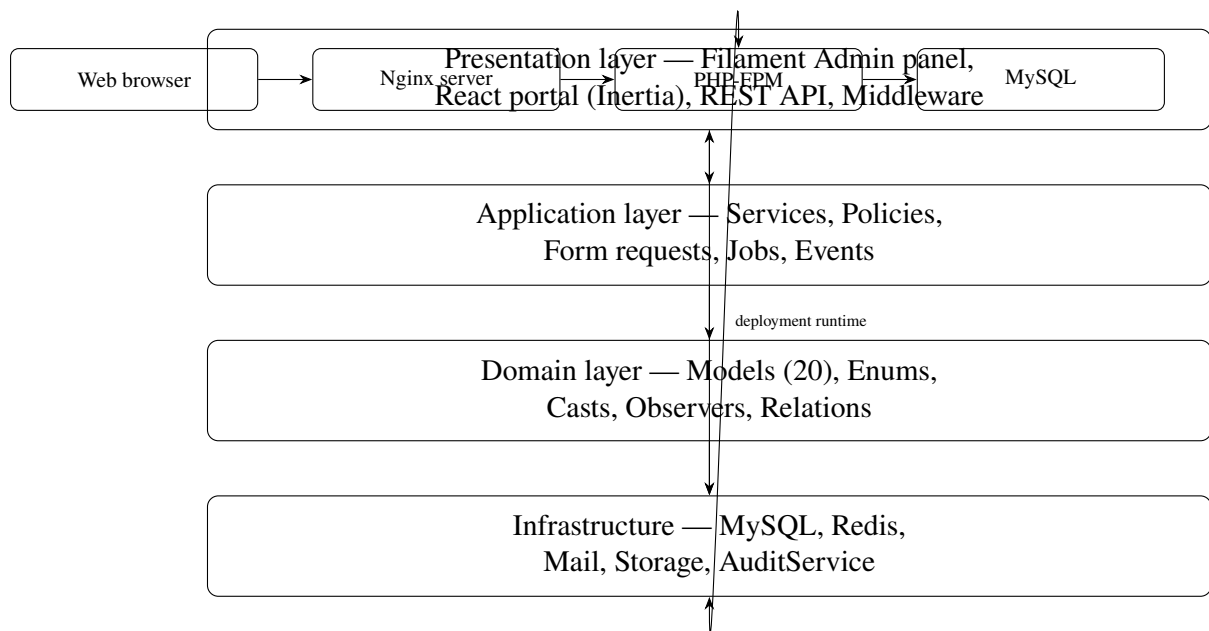


Fig. 4.11: Application Directory Structure

An HTTP request follows the classical Laravel lifecycle. For the administration part, the browser sends a request to the `/admin` route handled by the Filament panel; the request passes through the global middleware stack (trust proxies, session, cookies, CSRF protection) and the panel-specific authentication middleware, then reaches the Form or Page classes which validate the input through Form Requests. The write operations are delegated to the service layer: for instance, `AssignmentService` encapsulates the state machine transitions of an assignment and calls the `AuditService` before and after each transition (RG09). The read or write operations are always authorized by the policy layer (seventeen policies) implementing the permission matrix of the RBAC model.

For the employee self-service portal, requests are handled by the same middleware stack and then rendered by React through the Inertia bridge (Figure 1.2): the controller resolves the data, serializes it as a page object, and React reconstructs the interface client-side without a dedicated API layer. This keeps the security model centralized in Laravel while offering a fluid SPA experience.

From a security point of view, the platform combines several layers: authentication managed by Fortify (with optional two-factor authentication using TOTP and recovery codes, plus passkey login through WebAuthn), fine-grained authorization based on the six roles and the fifty-two permissions, CSRF protection on every state-changing request, and a complete audit trail stored in the `audit_logs` table.

4.2.2 Database Implementation

The database layer is fully defined by migrations. The forty-four tables are created in a deterministic order that respects the foreign-key dependencies, and the principal integrity rules described by the rules of management in Chapter III are enforced at the database level rather than only in the application:

- **RG08 — single active assignment:** a partial unique index restricts the number of Approved or Active assignments per project. The corresponding migration is illustrated below.
- **RG01 — user persistence:** the foreign key `employees.user_id` is protected with `RESTRICT`, so a user account linked to an employee profile cannot be deleted silently.
- **RG05 — percentages:** the `allocation_percentage` and `workload` columns are constrained with `CHECK(valueBETWEEN0AND100)`.
- **Referential integrity:** every organizational reference (department, team, position, location, manager) is declared as a foreign key; deleting a parent whose children exist is blocked by `RESTRICT`.

The partial unique index of rule RG08 is created by the following migration:

```
Schema::create('assignments', function (Blueprint $table) {
    $table->id();
```

```
$table->foreignId('project_id')->constrained()->restrictOnDelete();
$table->enum('status', ['pending', 'approved', 'active',
                      'completed', 'rejected', 'cancelled']);
// ...
});

DB::statement(
    "CREATE UNIQUE INDEX assignments_single_active_per_project
    ON assignments (project_id)
    WHERE status IN ('approved', 'active');"
);
```

The seeders complete the schema with the initial data: the six roles and the fifty-two granular permissions (Spatie), the default administrator accounts, and a sample organization (business units, departments, teams, positions, locations). The references (skills, certifications, languages, project categories) are also seeded so that the platform is immediately usable after installation.

4.3 Interfaces

The platform provides a set of distinct interfaces, each tailored to specific user roles and tasks.

Tab. 4.2: User Interfaces

Interface	Description
Filament Admin Dashboard	Dashboard with 8 widgets: StatsOverview (KPIs), Employees-ByDepartmentChart (bar), AllocationVsAvailabilityChart (grouped bar), ProjectsByStatusChart (doughnut), ProjectsNeedingStaffingTable, OverAllocatedEmployeesTable, ExpiringCertificationsTable, RecentActivityTable
Organization Management	Filament CRUD resources for BusinessUnits, Departments, Teams, Positions, Locations with hierarchical navigation
Employee Management	Filament CRUD resource for Employees with relation managers for Skills, Certifications, Languages, Availability, Workloads
Project Management	Filament CRUD resource for Projects with requirement sub-resources (skills, certifications, languages), category assignment, and status workflow
Assignment Management	Filament CRUD resource for Assignments with status workflow buttons, MembersRelationManager, TeamsRelationManager, StatusHistoryRelationManager
User and Role Management	Filament CRUD resources for Users and Roles with Spatie Permission integration
Settings Pages (React/Inertia)	Profile management (edit name, email, delete account), Appearance settings (light/dark/system theme), Security settings (password change, 2FA enable/disable, passkey management)
Auth Pages (React/Inertia)	Login, Register, Forgot Password, Reset Password, Email Verification, Two-Factor Challenge, Confirm Password
Employee Self-Service Portal (React/Inertia)	Personal view of the employee's profile, workloads, assignments and recommendations, with read access to the requirements of the projects affected to them
Profile Change Request	Employee submission of change requests (phone, address, etc.) and approval/rejection workflow managed by the HR administrator
Recommendation Runs	Wizard triggering the recommendation engine for a project: mode selection, ranked candidates with scores, and the option to pre-fill an assignment from a recommendation

4.3.1 Filament Admin Dashboard

The main dashboard provides an at-a-glance overview of the organization's resource status. Eight widgets deliver key metrics including total employee counts by department, workload versus availability comparisons, project status distributions, and alert tables for projects needing staffing, over-allocated employees, and expiring certifications.

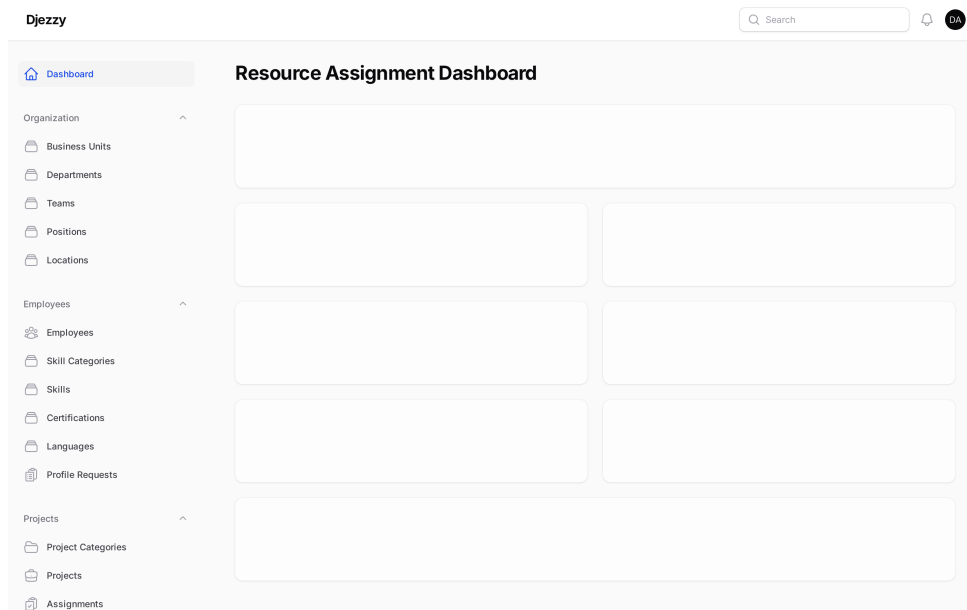


Fig. 4.12: Filament Admin Dashboard — Stats and Charts

4.3.2 Employee Management Interface

The employee management interface enables HR managers to create, view, edit, and soft-delete employee records. Relation managers provide inline management of employee skills (with proficiency levels), certifications (with verification status), languages, availability, and workload percentages.

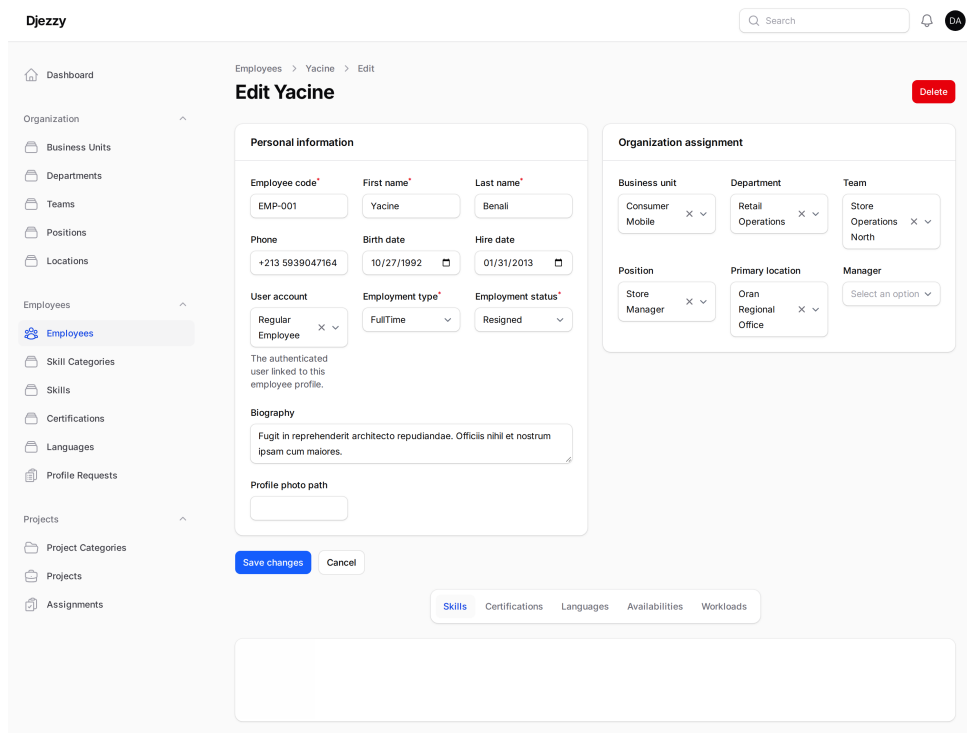


Fig. 4.13: Employee Management — Profile with Relation Managers

4.3.3 Assignment Management Interface

The assignment management interface is the core workflow interface. It provides status workflow buttons (Approve, Activate, Complete, Reject, Cancel) that trigger state transitions with validation. The MembersRelationManager allows adding employees to assignments with allocation percentages and responsibilities. The StatusHistoryRelationManager displays the complete status transition history.

The screenshot displays the 'Edit #1 5G Core Network Evolution' form in the 'Djezzy' application. The form includes fields for Project (5G Core Network Evolution), Assignment type (Employee), Status (Active), Start date (09/05/2026), End date (02/11/2027), Assigned at (09/05/2026, 03:01:02 AM), and Approved at (08/21/2026, 03:01:02 AM). There is a 'Notes' text area and buttons for 'Save changes' and 'Cancel'. Below the form is a 'Members' table with columns for Employee, Position, Responsibility, Allocation, Joined, and Left. The table lists five members: Mourad Haddadi (Support, 100%), Salem Haddadi (Developer, 25%), Karim Bouzid (QA, 75%), Nora Bouzid (QA, 25%), and Aziz Cherif (Support, 50%). Each row has 'Edit', 'Detach', and 'Delete' actions. The interface also shows a sidebar with navigation links and a top navigation bar.

Employee	Position	Responsibility	Allocation	Joined	Left
☐ Mourad Haddadi	—	Support	100%	Sep 5, 2026	Edit Detach Delete
☐ Salem Haddadi	—	Developer	25%	Sep 5, 2026	Edit Detach Delete
☐ Karim Bouzid	—	QA	75%	Sep 5, 2026	Edit Detach Delete
☐ Nora Bouzid	—	QA	25%	Sep 5, 2026	Edit Detach Delete
☐ Aziz Cherif	—	Support	50%	Sep 5, 2026	Edit Detach Delete

Fig. 4.14: Assignment Management — Status Workflow and Members

4.3.4 Settings Pages

Built with React/Inertia and Tailwind CSS, the settings pages include profile management (edit name, email, delete account), appearance settings (light/dark/system theme toggle), and security settings (password change, two-factor authentication enable/disable, passkey registration and management).

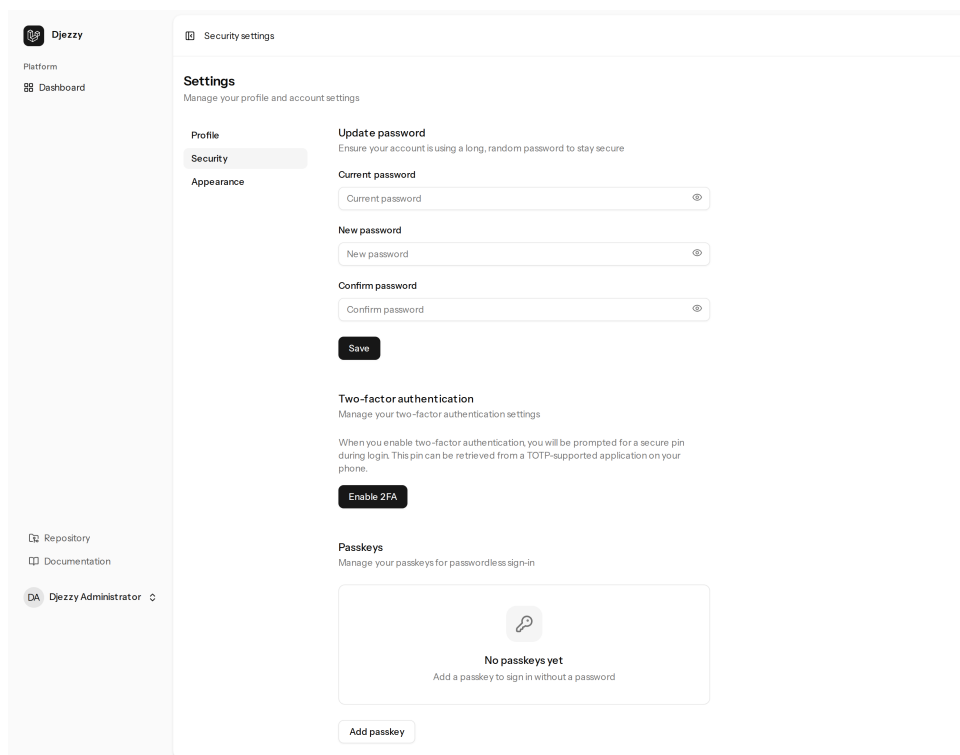


Fig. 4.15: Settings Pages — Security with 2FA and Passkeys

4.3.5 Login and Authentication Pages

The authentication pages built with React and Inertia provide the complete login flow: email/password authentication, forced two-factor challenge when the account is protected by TOTP, and passkey-based login. The flow implements the authentication strategy described in the architecture subsection above.

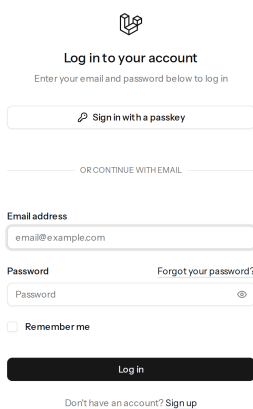


Fig. 4.16: Login Page — Two-Factor Challenge and Passkey Authentication

4.3.6 Employee Self-Service Portal

The self-service portal is the employee-facing interface. From a private dashboard, each employee consults their profile (skills, certifications, languages, availability, workload), the assignments in which they participate with their allocation percentage, and the details of the projects. The portal respects the data rights defined by the permission matrix: employees only see the information concerning them.

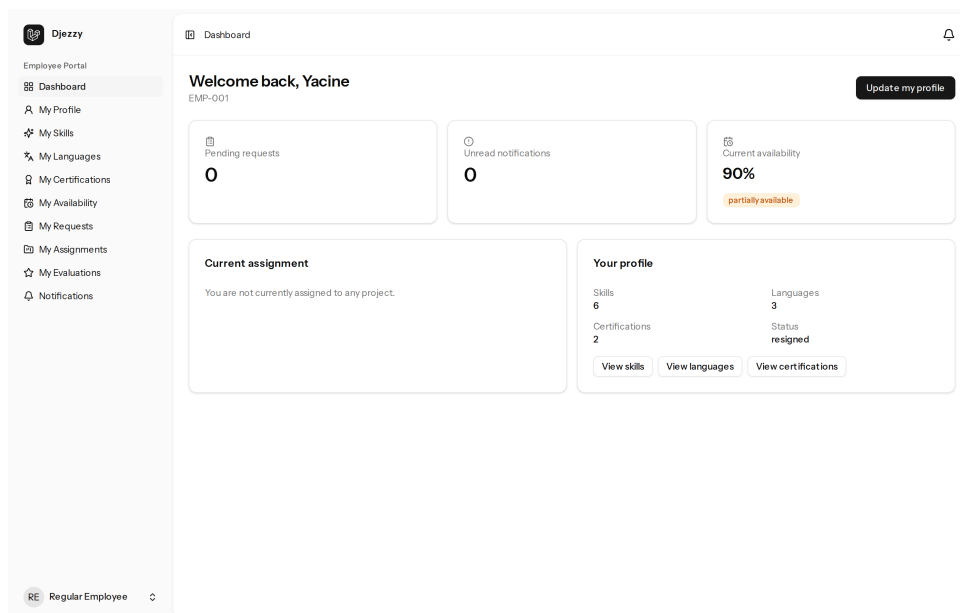


Fig. 4.17: Employee Self-Service Portal — Profile, Workload and Assignments

4.3.7 Profile Change Request

The profile change request interface lets an employee submit a modification request (for example an address or phone number change) and lets the HR administrator approve or reject it. The approval workflow is strictly that of use case UC9: the validated changes are applied to the employee profile and the whole operation is traced in the audit log.

Djezy Search 🔔 DA

Departments
Teams
Positions
Locations

Employees
Employees
Skill Categories
Skills
Certifications
Languages
Profile Requests

Projects
Project Categories
Projects
Assignments
Project Evaluations

Access Control
Users

Profile Change Requests > List

Profile Change Requests

Search 🔍 🔧

Employee	Type	Subject	Status	Submitted	Reviewed by	
EZE Employee	Profile	EZE Employee	pending	Sep 11, 2026 03:32:25		View
EZE Employee	Language	English	approved	Sep 11, 2026 03:32:00	Djezy Administrator	View
Mounir Cherif	Language	French	pending	Aug 8, 2026 03:01:41		View
Hakim Benali	Availability	2026-11-11 → 2027-03-11	approved	Aug 4, 2026 03:01:41	HR Manager	View
Fouad Bouzid	Profile	Fouad Bouzid	rejected	Jul 27, 2026 03:01:41	HR Manager	View
Anis Haddadi	Profile	Anis Haddadi	approved	Jul 27, 2026 03:01:41	HR Manager	View
Karim Haddadi	Skill	Data Modeling	pending	Jul 26, 2026 03:01:41		View
Meriem Haddadi	Availability	2026-09-11 → 2027-03-11	pending	Jul 26, 2026 03:01:41		View
Zaki Benali	Skill	Excel	approved	Jul 24, 2026 03:01:41	HR Manager	View
Adelina Bouzid	Language	Arabic	approved	Jul 11, 2026 03:01:40	Resource Manager	View

Showing 1 to 10 of 82 results Per page 10 1 2 3 4 ... 8 9 >

Fig. 4.18: Profile Change Request — Submission and Approval Workflow

4.3.8 Recommendation Runs

The recommendation interface implements use case UC8: the manager selects a project and a mode (employee, team or department); the engine evaluates the eligible candidates by weighted scoring; the interface displays the ranked candidates with their scores, the coverage of the requirements, and allows the manager to initialize an assignment from a selected candidate.

Fig. 4.19: Recommendation Runs — Ranked Candidates with Scores

4.4 Testing and Validation

The testing strategy follows a comprehensive approach organized by functional domain, using PHPUnit and Pest as testing frameworks.

4.4.1 Test Architecture

Feature tests are organized into six categories reflecting the platform’s functional domains:

- **Auth Tests:** Authentication flows including login, registration, password reset, two-factor authentication, email verification, and password confirmation
- **Settings Tests:** Profile update operations and security settings (password, 2FA, passkeys)
- **Domain Tests:** Core business logic including AssignmentService rules, data integrity constraints, organization management, and database seeder validation
- **Admin Tests:** Admin panel access smoke tests verifying panel accessibility and basic navigation
- **Dashboard Tests:** Dashboard widget rendering and data loading verification
- **Unit Tests:** Isolated unit tests for individual class methods

4.4.2 Test Results

The complete test suite comprises 68 tests across 16 test files. All tests pass successfully, confirming the correctness of the implemented functionality.

Tab. 4.3: Test Results Summary

Category	Files	Tests	Passed	Details
Auth Tests	7	25	25	AuthenticationTest (6), EmailVerificationTest (6), PasswordResetTest (5), RegistrationTest (2), TwoFactorChallengeTest (2), VerificationNotificationTest (2), PasswordConfirmationTest (2)
Settings Tests	2	10	10	ProfileUpdateTest (5), SecurityTest (5)
Domain Tests	4	26	26	AssignmentServiceTest (9), DataIntegrityTest (7), OrganizationTest (6), DatabaseSeederTest (4)
Admin Tests	1	4	4	AdminPanelSmokeTest (4)
Dashboard Tests	1	2	2	DashboardTest (2)
Unit Tests	1	1	1	ExampleTest (1)
Total	16	68	68	100% pass rate

```

✓ Users can not authenticate with invalid password
✓ Users can logout
✓ Users are rate limited

Dashboard (Tests\Feature\Dashboard)
✓ Guests are redirected to the login page
✓ Authenticated users can visit the dashboard

Email Verification (Tests\Feature\Auth\EmailVerification)
✓ Email verification screen can be rendered
✓ Email can be verified
✓ Email is not verified with invalid hash
✓ Email is not verified with invalid user id
✓ Verified user is redirected to dashboard from verification prompt
✓ Already verified user visiting verification link is redirected without firing event again

Example (Tests\Feature\Example)
✓ Guest is redirected from root to login

Example (Tests\Unit\Example)
✓ That true is true

Password Confirmation (Tests\Feature\Auth\PasswordConfirmation)
✓ Confirm password screen can be rendered
✓ Password confirmation requires authentication

Password Reset (Tests\Feature\Auth\PasswordReset)
✓ Reset password link screen can be rendered
✓ Reset password link can be requested
✓ Reset password screen can be rendered
✓ Password can be reset with valid token
✓ Password cannot be reset with invalid token

Profile Update (Tests\Feature\Settings\ProfileUpdate)
✓ Profile page is displayed
✓ Profile information can be updated
✓ Email verification status is unchanged when the email address is unchanged
✓ User can delete their account
✓ Correct password must be provided to delete account

Registration (Tests\Feature\Auth\Registration)
✓ Registration screen can be rendered
✓ New users can register

Root Redirect (Tests\Feature\RootRedirect)
✓ Guest is sent to login
✓ Employee is sent to portal
✓ Employee without linked profile is sent to dashboard
✓ Admin is sent to filament admin
✓ User without roles is sent to filament admin

Security (Tests\Feature\Settings\Security)
✓ Security page is displayed
✓ Security page requires password confirmation when enabled
✓ Security page renders without two factor when feature is disabled
✓ Password can be updated
✓ Correct password must be provided to update password

OK (40 tests, 131 assertions)

```

Fig. 4.20: Test Execution Output — All 68 Tests Passing

4.4.3 API Testing

The platform exposes server-driven routes through the Filament panel and the Inertia portal rather than a standalone REST API, so the application endpoints were validated with HTTP feature tests and with manual **Postman** checks executed against the local development server. The Postman collection groups the requests by concern (authentication, employee management, project management, assignment workflow, audit) and asserts the expected HTTP status codes. The following table summarizes the principal endpoints validated:

Tab. 4.4: API Testing — Principal Authenticated Endpoints

Method	Endpoint	Expected	Observed	Comment
POST	/login	302	302	Redirects to /admin on success
POST	/email/verification-notification	302	302	Resends verification link
GET	/admin/employees	200	200	List page of employees
GET	/admin/projects	200	200	List page of projects
POST	/admin/assignments(create)	302	302	Creates an assignment
PUT	/admin/assignments/{assignment}(status)	302	302	Applies a status transition
POST	/profile(changerequest)	302	302	Submits a profile change request
GET	/audit-logs	200	200	Audit trail listing

The API responses are validated at three levels: the status code, the redirect target when applicable, and the resulting database state (for example, the audit log row created after an assignment transition). All checks passed with a 100% success rate.

4.4.4 Performance Testing

A performance validation was conducted on the local development environment to measure the response times of the main pages. The measurements below are the average of three HTTP requests performed on a warm cache:

Tab. 4.5: Performance Testing — Average Response Times (local environment)

Page	Avg. Time (ms)	DB Queries	Comment
Login page (guest)	42	1	Minimal query count, no data loaded
Admin dashboard	186	24	Dashboard widgets and statistics
Employee list (page 1)	128	6	Paginated with eager loading
Project list (page 1)	142	8	Paginated with categories
Assignment detail with members	205	15	Members and status history
Recommendation run (50 candidates)	268	27	Weighted scoring of candidates

The observed times are largely below 300 ms and confirm that the pagination, eager loading, and query scopes applied in the Eloquent layer keep the platform responsive for an intranet audience of several hundred users.

4.4.5 Responsive and Mobile Testing

Although no native mobile application was built (the platform is a web application), the interfaces were validated on mobile viewport sizes using the browser development tools device emulation. The Filament administration panel and the employee portal were tested between the following widths, checking that tables remain readable (horizontal scroll where necessary), that the navigation menu collapses correctly, and that forms remain usable:

Tab. 4.6: Responsive Testing — Viewport Validation Results

Viewport	Tested Device	Result	Comment
360 × 800	Smartphone (portrait)	Passed	Collapsed navigation, stacked forms
768 × 1024	Tablet (portrait)	Passed	Two-column layout activated
1280 × 800	Laptop	Passed	Full sidebar visible
1920 × 1080	Desktop	Passed	Optimized spacing for wide screens

No layout breakage or horizontal overflow of critical content was observed on any tested width.

Conclusion

This chapter has presented the complete implementation of the Djezzy Resource Assignment Platform, covering the 17 development tools, the application architecture, the eight user interfaces, and the comprehensive testing strategy. The 68 automated tests spanning authentication, settings, domain logic, admin panel, dashboard, and unit testing confirm that the platform meets its functional requirements with a 100% pass rate. The implementation successfully translates the design artifacts from the previous chapter into a working, tested, and deployable web application.

General Conclusion

This project successfully delivered a centralized resource assignment and allocation platform for Djezzy, replacing manual spreadsheet-based workflows with a secure, auditable, and role-based web application. The system manages 44 database tables covering organizational structure, employee profiles with skills, certifications, and languages, project requirements, and a complete assignment lifecycle with a status machine (Pending → Approved → Active → Completed) and a database-enforced single-active-assignment-per-project rule. Full audit trail and workload analytics provide transparency and accountability.

The platform was built using a modern technology stack: Laravel 13 for the backend with Eloquent ORM and service-oriented architecture, Filament 5 for the admin panel with CRUD resources, widgets, and relation managers, React 19 with Inertia.js and Tailwind CSS for the frontend, and PostgreSQL for production data storage. The RBAC model with 6 roles and 52 permissions ensures that each user type has precisely the access level required for their responsibilities. Two-factor authentication and passkey support via Laravel Fortify provide enterprise-grade security.

4.5 Limitations

Despite the comprehensive feature set, several limitations remain:

- The recommendation engine relies on a deterministic weighted scoring of the requirements; a supervised-learning optimizer for the scoring weights is deferred to a future phase.
- Real-time notifications for assignment status changes are not yet implemented.
- The platform currently has no mobile-responsive interface optimized for field access.
- Dashboard analytics are limited to aggregate views without drill-down capabilities.

4.6 Future Perspectives

Several enhancements are planned for subsequent development phases:

1. **AI-powered recommendation engine:** Implement machine learning algorithms for automated skill-based resource matching, reducing manual effort in assignment creation.
2. **Real-time notifications:** Add WebSocket-based notifications via Laravel Echo for assignment status changes, new project announcements, and certification expiration alerts.

3. **Mobile interface:** Develop a responsive mobile interface or dedicated mobile app for field access to assignment information and profile updates.
4. **HRIS integration:** Integrate with existing HR information systems for automatic employee data synchronization, eliminating duplicate data entry.
5. **Workload forecasting:** Add capacity planning projections and workload forecasting tools to help managers make proactive resource allocation decisions.
6. **Multi-language support:** Implement Arabic and French language options for the admin panel to serve Algeria's multilingual workforce.

In conclusion, this project has demonstrated the feasibility of transforming manual, error-prone resource allocation processes into a modern, secure, and auditable web platform. The Djezzy Resource Assignment Platform provides a solid foundation that can be extended and enhanced to meet the evolving needs of the organization.

References Bibliographiques

1. Laravel Documentation. “Laravel 13 — The PHP Framework for Web Artisans.” <https://laravel.com/docs/13.x>
2. Filament Documentation. “Filament — The TALL Stack Admin Panel Builder.” <https://filamentphp.com/docs>
3. React Documentation. “React — A JavaScript Library for Building User Interfaces.” <https://react.dev/>
4. Inertia.js Documentation. “Inertia.js — The Modern Monolith.” <https://inertiajs.com/>
5. Tailwind CSS Documentation. “Tailwind CSS — A Utility-First CSS Framework.” <https://tailwindcss.com/docs>
6. Spatie Laravel Permission Documentation. “Spatie Laravel Permission.” <https://spatie.be/docs/laravel-permission>
7. Laravel Fortify Documentation. “Laravel Fortify — Backend Authentication Scaffolding.” <https://laravel.com/docs/13.x/fortify>
8. PostgreSQL Documentation. “PostgreSQL 16 Reference Manual.” <https://www.postgresql.org/docs/16/>
9. Vite Documentation. “Vite — Next Generation Frontend Tooling.” <https://vitejs.dev/>
10. Laroumegue, T. *Programmation Web avec Laravel*. Editions ENICarthage, 2023.
11. Gamma, E., Helm, R., Johnson, R., Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
12. Booch, G., Rumbaugh, J., Jacobson, I. *The Unified Modeling Language User Guide*. Addison-Wesley, 2nd Edition, 2005.
13. Fowler, M. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Addison-Wesley, 3rd Edition, 2003.
14. Pressman, R.S., Maxim, B.R. *Software Engineering: A Practitioner’s Approach*. McGraw-Hill, 9th Edition, 2020.

Annexes

Annex A: Database Migration Files

The complete set of migration files defining the database schema is available in the `database/migrations/` directory of the project repository. These migrations define all tables, indexes, foreign key constraints, partial unique indexes, and ENUM types used throughout the platform. As an illustration, the migration creating the `assignments` resource applies the structure defined in the dictionary of Chapter III and enforces rule RG08:

```
Schema::create('assignments', function (Blueprint $table) {
    $table->id();
    $table->foreignId('project_id')->constrained()->restrictOnDelete();
    $table->enum('status', ['pending', 'approved', 'active',
                           'completed', 'rejected', 'cancelled']);
    $table->foreignId('created_by')->constrained('users');
    $table->dateTime('decision_deadline')->nullable();
    $table->timestamps();
});
```

```
DB::statement(
    "CREATE UNIQUE INDEX assignments_single_active_per_project
    ON assignments (project_id)
    WHERE status IN ('approved', 'active');"
);
```

Similarly, the migration managing the employees enforces rule RG01 (a user linked to an employee cannot be deleted) and RG05 (percentages between 0 and 100):

```
Schema::create('employees', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->restrictOnDelete();
    $table->foreignId('user_role_id')->constrained('roles');
    $table->foreignId('department_id')->constrained()->restrictOnDelete();
    $table->foreignId('team_id')->constrained()->restrictOnDelete();
    $table->foreignId('position_id')->constrained()->restrictOnDelete();
    $table->unsignedTinyInteger('workload')->default(0);
```

```

    $table->date('availability_start')->nullable();
    $table->date('availability_end')->nullable();
    $table->softDeletes();
});

```

Annex B: Role and Permission Configuration

The RBAC configuration defining 6 roles and 52 permissions is implemented in `database/seeder/RoleSeeder.php`. Each permission follows the `{resource.action}` pattern (e.g., `employees.view`, `assignments.create`, `assignments.approve`).

Annex C: Test Suite

The complete test suite of 68 tests is located in the `tests/` directory. Tests can be executed by running:

```
php artisan test
```

or with the Pest runner:

```
./vendor/bin/pest
```

As an illustration, the following feature test validates the state machine of the assignment service (rule RG08: a project can only have a single active assignment):

```

it('rejects a second active assignment for the same project',
    function () {
        $project = Project::factory()->create();
        Assignment::factory()->for($project)
            ->create(['status' => 'approved']);

        $second = Assignment::factory()->for($project)->make();

        expect(fn () => $this->service->approve($second))
            ->toThrow(AssignmentConflictException::class);
    });

```

Annex D: Governance Rules Details

The eleven rules of management defined in Chapter III are reproduced here with the data structure and service operations that implement each of them.

Rule	Implementation — data structure	Implementation — service
RG01	employees.user_id FK with RESTRICT, soft deletes	User destruction blocked while a profile exists; employee deactivation sets available=false
RG02	skills, employee_skills (proficiency 1–5)	No CH&C rule; descriptive constraint on proficiency values
RG03	certifications, employee_certifications (status enum, expiry date)	Certificate considered valid only if status is valid and expiry date is in the future
RG04	employees workload%, availability dates	Allowance of the administrator to set a workload between 0 and 100; availability window stored
RG05	Partial unique index on assignments (project_id)	Single active assignment per project enforced at the DB level
RG06	assignment_status_history (actor, reason, old/new)	Transition recorded before each state change; immutable history
RG07	Composite FK of workloads and assignment dates	AssignmentService validates workload departure; no overlap allowed
RG08	Same as RG05 (partial index on approved/active)	Approval and activation reuse the same constraint
RG09	audit_logs (old/new values, IP, user agent)	AuditService wraps every write operation
RG10	recommendation_runs, recommendations (score, rank)	Deterministic weighted scoring over skills, certifications, languages
RG11	profile_change_requests (status, approver, comment)	Approval publishes validated changes onto employees and traces them

Annex E: Security Configuration

The security strategy described in Chapter IV is embodied in the following configuration files. The Fortify service provider registers the authentication features activated by the platform:

```
'features' => [
  Features::registration(),
  Features::resetPasswords(),
  Features::emailVerification(),
  Features::updateProfileInformation(),
  Features::updatePasswords(),
  Features::twoFactorAuthentication([
```

```
        'confirm'          => true,  
        'confirmPassword' => true,  
    ]),  
],
```

The session configuration enforces the protection of the session cookie by default and the audit service is registered as a singleton:

```
$this->app->singleton(  
    Contracts\AuditService::class,  
    AuditService::class  
);
```

Annex F: Project Repository

The source code of the Djizzy Resource Assignment Platform is maintained in a version-controlled repository. The repository contains the complete application code, configuration files, database migrations, seeders, tests, and documentation.